Context Engineering Harness

Complete Guide

A beginner-friendly, code-accurate walkthrough of an AI document question-answering application

Document	CONTEXT_ENGINEERING_HARNESS_COMPLETE_GUIDE.pdf
Scope	Backend (FastAPI), frontend (React), RAG pipeline, OCR, tables, images, embeddings, Pinecone
Source of truth	The actual repository as read on the date below
Generated	2026-10-11
Chapters	22

This guide was produced without modifying any application source code. It describes implemented, partially implemented, and unimplemented features honestly.

Table of Contents

Chapter 1. Project Overview 12

1.1 What is the Context Engineering Harness? . . . 12

1.2 What problem is it intended to solve? . . . 12

1.3 Why is this project useful? . . . 12

1.4 Who might use it? . . . 12

1.5 What can the user currently do with it? . . . 12

1.6 Technologies used, and why . . . 13

1.7 What is implemented, partly implemented, or only planned . . . 14

Chapter 2. Complete Project Directory Structure 16

2.1 The actual tree . . . 16

2.2 Root-level entries . . . 16

2.3 Backend files, one by one . . . 17

2.4 Frontend files, one by one . . . 21

2.5 Storage directories . . . 21

2.6 What does NOT exist . . . 21

Chapter 3. Complete System Architecture 23

3.1 The big picture . . . 23

3.2 Arrow-by-arrow explanation . . . 23

3.3 Upload and indexing architecture 24

3.4 Retrieval and answer architecture (/documents/ask) 24

3.5 Where the vision model sits 25

Chapter 4. Application Startup and Execution 26

4.1 How the backend starts 26

4.2 What Uvicorn does 26

4.3 How FastAPI creates the application 26

4.4 How CORS is configured (and why) 26

4.5 How imports connect the modules 27

4.6 How the React frontend starts 27

4.7 How the browser talks to FastAPI 27

4.8 What happens when the user sends a message 27

4.9 What happens when the user uploads a PDF 28

Chapter 5. Complete Code Walkthrough 29

5.1 schemas.py - defining the shape of data 29

5.2 llm.py - calling the language models 29

5.3 main.py - the endpoints 30

5.4 document_service.py - the orchestrator 32

5.5 text_extractor.py - selectable text 32

5.6 ocr.py - reading page images 32

5.7 table_extractor.py - structured tables . . . 33

5.8 image_extractor.py - saving pictures . . . 33

5.9 chunker.py - one shape for everything . . . 34

5.10 embedding_service.py - text becomes numbers . . . 34

5.11 pinecone_service.py - the vector database . . . 35

5.12 rag_service.py - the retrieval glue . . . 36

5.13 engine.py - the Context Engine . . . 36

5.14 App.jsx - the frontend . . . 37

Chapter 6. Function Calling Map: Who Calls Whom 39

6.1 Chat request call graph . . . 39

6.2 PDF upload call graph . . . 39

6.3 Document question call graph . . . 40

6.4 Semantic search call graph . . . 41

6.5 A flat direct/indirect map . . . 41

Chapter 7. Chat Pipeline Explained 43

7.1 The React state behind the chat . . . 43

7.2 How the request is built and sent . . . 43

7.3 How the backend validates the input . . . 43

7.4 How the endpoint calls the model . . . 43

7.5 A complete worked example . . . 43

7.6 Loading and error states 44

7.7 What happens when the LLM fails 44

Chapter 8. PDF Upload and Document Processing 45

8.1 The complete lifecycle of one PDF 45

8.2 Upload validation (two layers) 45

8.3 Document IDs 45

8.4 The difference between the six artefacts 45

8.5 One page producing several content units 46

8.6 How ids and page numbers connect content to its source 47

Chapter 9. OCR Explained 48

9.1 What OCR means 48

9.2 Why ordinary PDF text extraction sometimes fails 48

9.3 How the code decides which pages need OCR 48

9.4 Rendering a page as an image 48

9.5 What Tesseract does 48

9.6 How OCR output becomes a text unit and continues downstream 48

9.7 Limitations of the current OCR implementation 49

Chapter 10. Table Extraction Explained 50

10.1 Why tables need special handling 50

10.2 How the current extractor works 50

10.3 Headers and rows 50

10.4 How a table becomes searchable text 50

10.5 The same table at every stage 50

10.6 Why both forms matter 51

Chapter 11. Image Extraction and Image Storage 52

11.1 How images are extracted 52

11.2 What an image reference is 52

11.3 Where image files are stored, and how the identifiers relate 52

11.4 Why an image file is not the same as its metadata 52

11.5 How an image becomes a content unit 52

11.6 How a reference becomes a real path 53

11.7 Limitations of the current extraction 53

Chapter 12. Embeddings Explained from Zero 54

12.1 What a vector is 54

12.2 What an embedding is 54

12.3 Why text is converted into numbers 54

12.4 Semantic similarity vs. exact keyword matching 54

12.5 Dimensions, and why query and documents must match 54

12.6 The actual Gemini integration 54

12.7 How text, tables and images are embedded (the honest version) 54

12.8 How a vector is associated with a content unit 55

Chapter 13. Pinecone Explained 56

13.1 Why a vector database is used 56

13.2 Index, namespace, ids, values, metadata 56

13.3 How upsert works 56

13.4 How similarity search works 56

13.5 Document filters 56

13.6 What results contain, and how the real content is retrieved 56

13.7 Index configuration requirements 57

13.8 Migration / re-indexing note 57

Chapter 14. Retrieval-Augmented Generation (RAG) 58

14.1 RAG from absolute beginner level 58

14.2 The actual implementation 58

14.3 One question, followed all the way 58

14.4 How text, tables and images are treated 58

14.5 Finding an image vs. sending an image to a model 59

14.6 What remains incomplete about image-to-LLM integration 59

Chapter 15. Context Engine Explained 60

15.1 Why a context engine is useful 60

15.2 Retrieval vs. context assembly . . . 60
15.3 What assemble_context() actually does . . . 60
15.4 What format_context() actually does . . . 60
15.5 How image references are represented . . . 60
15.6 How the Context Engine connects to RAG and the LLM . . . 60
15.7 What the current context dictionary contains . . . 61
15.8 Implemented vs. planned context components . . . 61
15.9 Why one authoritative context path is a good principle . . . 61

Chapter 16. Frontend Explained 62

16.1 Components and imports . . . 62
16.2 React state . . . 62
16.3 Controlled inputs . . . 62
16.4 Chat messages, loading indicators and errors . . . 62
16.5 PDF selection and upload validation . . . 62
16.6 FormData and the upload request . . . 62
16.7 API responses and how the UI renders them . . . 63
16.8 CSS layout and responsiveness . . . 63
16.9 Why the frontend never touches Pinecone or Gemini directly . . . 63

Chapter 17. API Reference 64

17.1 GET / . . . 64

17.2 POST /chat 64

17.3 POST /documents/upload 64

17.4 POST /documents/extract-preview 65

17.5 POST /documents/search 65

17.6 POST /documents/ask 66

17.7 Endpoints that do NOT exist 66

Chapter 18. Configuration, Dependencies, and Environment 67

18.1 The Python virtual environment 67

18.2 requirements.txt 67

18.3 Environment variables 67

18.4 Where .env must be placed, and why 68

18.5 Gemini, Pinecone and Ollama configuration 68

18.6 Frontend dependencies 68

18.7 Local storage paths 68

Chapter 19. Testing and Debugging 69

19.1 What tests exist 69

19.2 How to run them 69

19.3 Verifying each capability by hand 69

19.4 Debugging guide for common failures 70

19.5 What is missing from testing 71

Chapter 20. End-to-End Execution Traces 72

20.1 Sending a normal chat message 72

20.2 Uploading a text-based PDF 72

20.3 Uploading a scanned PDF requiring OCR 72

20.4 Uploading a PDF containing a table 72

20.5 Uploading a PDF containing an image 73

20.6 Asking a question that triggers RAG 73

20.7 Retrieving an image and sending it to a vision model 73

Chapter 21. Current Limitations and Future Roadmap 74

21.1 Status table 74

21.2 Current functionality (verified by reading the code) 75

21.3 Possible future improvements 75

Chapter 22. Glossary and Learning Revision 76

22.1 Beginner-friendly glossary 76

22.2 How the entire project works (short summary) 77

22.3 The function names to remember 77

22.4 Which file handles which responsibility 78

22.5 The complete request-to-response flow (revision) 78

22.6 Final honesty notes 79

Chapter 1. Project Overview

1.1 What is the Context Engineering Harness?

The **Context Engineering Harness** is a small full-stack AI application. It has three visible pieces: a browser interface built with React, a backend web service built with Python and FastAPI, and a set of AI services that the backend talks to (a local language model through Ollama, an embedding model and vector database through Google Gemini and Pinecone, and an optical character recognition tool called Tesseract).

The word *harness* is important. A harness in software is a controlled framework that lets you try things and observe what happens. This project is a place to **experiment with feeding different kinds of information into a language model**. Specifically, it explores how to turn a PDF document - which can contain plain text, tables, and images - into a form that a language model can use to answer questions.

1.2 What problem is it intended to solve?

A standard language model only knows what it learned during training. It does not know the contents of a document you upload today. If you ask it about a private report, it will either say it does not know or, worse, invent an answer. The problem this project addresses is: **how do we let a language model answer questions about our own documents, including their tables and images?**

The general technique for this is called **Retrieval-Augmented Generation**, or **RAG** for short. You will meet that term many times in this guide. In simple words, RAG means: first *find* the relevant pieces of the document, then *add* those pieces to the question you give the model, and finally let the model *generate* an answer using them. The project also experiments with a second idea: sending the actual images from the document to a model that can see pictures (a **vision-capable** or **multimodal** model).

1.3 Why is this project useful?

- It is a learning vehicle. Every stage of a real retrieval pipeline is implemented in small, readable files, so you can follow data from an uploaded PDF all the way to a model's answer.
- It treats **multimodal** content (text, tables, images) as first-class citizens instead of ignoring everything that is not plain text.
- It separates concerns: extraction, chunking, embedding, storage, retrieval, and context assembly each live in their own module.
- It is honest about the difference between *finding* an image and *understanding* an image, which is a common source of confusion in RAG systems.

1.4 Who might use it?

1.5 What can the user currently do with it?

Based on the actual code, the React interface exposes exactly two user actions: sending a chat message and uploading a PDF. The backend exposes a few more endpoints than the interface currently uses.

User / client action	What actually happens today	Implemented ?
Type a message and press Send	The browser calls `POST /chat`. The backend sends the text to the local Ollama model **llama3.2** and returns the reply.	Yes
Select a PDF and press Upload	The browser calls `POST /documents/upload`. The backend extracts text, runs OCR where needed, extracts tables and images, builds content units, embeds them, and stores them in Pinecone.	Yes
Ask a question about an uploaded document (through the interface)	There is **no** button in the React interface that calls the document-question endpoint. The endpoint exists, but the interface does not use it yet.	No (backend only)
Semantic search over documents	The endpoint `POST /documents/search` exists and works, but no interface calls it.	Backend only
Look inside the extraction result	The endpoint `POST /documents/extract-preview` exists for development inspection. No interface calls it.	Backend only
Conversation memory / persistence	Chat messages live only in React state and disappear on refresh. Nothing is stored server-side.	Not implemented

1.6 Technologies used, and why

Technology	Where it is used	Why it is used (purpose in this project)
Python 3	Backend	The language of the whole server, extraction, and AI orchestration.
FastAPI	Backend web framework	Turns Python functions into HTTP endpoints and validates JSON automatically.
Uvicorn	Backend server	The web server that actually runs the FastAPI application.
Pydantic	Backend	Defines and validates the shape of request and response data.
React 19	Frontend	Builds the interactive chat and upload interface in the browser.
Vite	Frontend build tool	Runs the development server and builds the frontend.
Ollama	LLM runtime	Runs language models locally on the machine.
llama3.2	Ollama model	The text-only chat model used by `/chat`.
qwen3.5:4b	Ollama model	The vision-capable (multimodal) model used when images are retrieved.

Technology	Where it is used	Why it is used (purpose in this project)
PyMuPDF (fitz)	Extraction	Opens PDFs, reads text, renders pages to images, and extracts embedded images.
pdfplumber	Extraction	Detects and reads tables in PDFs.
Tesseract + pytesseract	OCR	Reads text from page images when the PDF has no selectable text.
Pillow (PIL)	Image handling	Converts rendered page pixels into an image object for Tesseract.
Google Gemini	Embeddings	Turns text into vectors using `gemini-embedding-001`.
Pinecone	Vector database	Stores and searches vectors, with metadata filters.
python-dotenv	Configuration	Loads API keys and settings from the `.env` file.

1.7 What is implemented, partly implemented, or only planned

Capability	Status	Evidence in the code
Plain text chat with an LLM	Implemented	`main.py` **chat()** → `llm.py` **generate_response()**
PDF upload and storage	Implemented	`document_service.py` **save_document()**
Selectable-text extraction	Implemented	`text_extractor.py` **extract_text_from_pdf()**
OCR fallback for scanned pages	Implemented	`ocr.py` **extract_text_with_ocr()**
Table extraction	Implemented	`table_extractor.py` **extract_tables_from_pdf()**
Embedded image extraction	Implemented	`image_extractor.py` **extract_images_from_pdf()**
Unified content units	Implemented	`chunker.py` **build_content_units()**
Text / table embeddings	Implemented	`embedding_service.py` **embed_content_units()**
True visual (image) embeddings	NOT implemented	Images are embedded from a **text description of their metadata**, not from pixels. See Chapter 12.
Pinecone indexing and search	Implemented	`pinecone_service.py` **upsert_content_units()** / **search_similar_content()**
Context assembly	Implemented (minimal)	`engine.py` **assemble_context()** returns a small fixed dictionary
Document Q&A through the UI	Not implemented	Endpoint `/documents/ask` exists; the React app never calls it

Capability	Status	Evidence in the code
Sending real images to a vision model	Implemented at the backend level	`llm.py` **generate_multimodal_response()** with **qwen3.5:4b**
Conversation persistence / memory	Not implemented	**assemble_context()** sets `conversation=None`, `memory=[]`
Knowledge graph (e.g. Neo4j)	Not implemented	**assemble_context()** sets `knowledge_graph=[]`
Evaluation tooling	Not implemented	No evaluation code exists in the repository

> **Status note:** Where this guide says “not implemented” it means there is no code for that feature in the repository. Where it says “backend only” it means the code exists and runs, but the React interface does not currently trigger it.

Chapter 2. Complete Project Directory Structure

2.1 The actual tree

This is the real directory layout of the repository, ignoring generated and third-party folders such as `node_modules`, `.venv`, and `__pycache__`.

```
context-engineering-harness/          (project root)
|-- .env                              API keys and settings (NOT committed / secret)
|-- .gitignore
|-- backend/
|   |-- requirements.txt              pinned Python dependencies
|   |-- test_embedding.py             manual script: embedding smoke test
|   |-- test_indexing.py              manual script: upsert one vector
|   |-- test_multimodal_rag.py        manual integration test: retrieval + images
|   |-- test_pinecone.py              manual script: index statistics
|   |-- test_retrieval.py             manual script: similarity search
|   |-- test_visual_embedding.py      manual script (STALE - see note below)
|   |-- app/
|   |   |-- __init__.py               marks "app" as a Python package (empty)
|   |   |-- main.py                   FastAPI app + all HTTP endpoints
|   |   |-- schemas.py                Pydantic request/response models
|   |   |-- llm.py                    Ollama text + multimodal model calls
|   |   |-- context_engine/
|   |   |   |-- __init__.py           (empty)
|   |   |   |-- engine.py             assemble_context / format_context / resolve_image_path
|   |   |-- rag/
|   |       |-- __init__.py           (empty)
|   |       |-- document_service.py   save_document + process_document (orchestrator)
|   |       |-- text_extractor.py     selectable-text extraction + OCR page detection
|   |       |-- ocr.py                Tesseract OCR fallback
|   |       |-- table_extractor.py    pdfplumber table extraction
|   |       |-- image_extractor.py    embedded image extraction + storage
|   |       |-- chunker.py            build unified "content units"
|   |       |-- embedding_service.py  Gemini embedding generation
|   |       |-- pinecone_service.py   Pinecone upsert + similarity search
|   |       |-- rag_service.py        retrieve_context (embed query + search)
|   |-- storage/
|       |-- documents/                saved original PDFs, named .pdf
|       |-- images/                   extracted images, one sub-folder per document
|-- frontend/
|   |-- index.html                    HTML page that loads React
|   |-- package.json                  frontend dependencies and scripts
|   |-- package-lock.json             exact locked frontend dependency versions
|   |-- vite.config.js                Vite configuration (React plugin)
|   |-- .oxlintrc.json                linter configuration
|   |-- README.md                     default Vite/React template README
|   |-- public/
|   |   |-- favicon.svg, icons.svg
|   |-- src/
|       |-- main.jsx                  React entry point (mounts )
|       |-- App.jsx                   the entire user interface + fetch calls
|       |-- App.css                   layout/styling for App.jsx
|       |-- index.css                 global/base styles from the Vite template
|       |-- assets/                   (template assets)
```

Figure 2.1 - actual repository layout

Stale file: `backend/test_visual_embedding.py` imports a function named `generate_image_embedding` from `app.rag.embedding_service`. That function does **not** exist in the current `embedding_service.py`. Running this script fails immediately with an **ImportError**. It appears to be left over from an earlier experiment with true visual embeddings. It is documented here honestly rather than corrected, because the task forbids changing code.

2.2 Root-level entries

Entry	Type	Responsibility
.env	file	Holds secrets and settings. Keys present: `GEMINI_API_KEY`, `PINECONE_API_KEY`, `PINECONE_INDEX_NAME`. Values are never shown in this guide.
.gitignore	file	Tells Git which files not to track (for example local environments and secrets).
backend/	folder	The Python/FastAPI server and all document-processing logic.
frontend/	folder	The React browser application.

2.3 Backend files, one by one

For each file: what it is, why it exists, what it imports, who calls it, what it calls, what goes in, what comes out, and who depends on it.

app/main.py - the entry point and all HTTP endpoints

Aspect	Detail
Imports	`fastapi` (FastAPI, File, HTTPException, UploadFile); `CORSMiddleware`; schemas; `generate_response`, `generate_multimodal_response`, `LLMError`; `save_document`, `process_document`; `generate_embedding`; `search_similar_content`; `assemble_context`, `format_context`, `resolve_image_path`
Why it exists	It is the single place where HTTP requests become Python function calls.
Calls	The LLM module, the document service, the embedding service, the Pinecone service, and the context engine.
Called by	Uvicorn (the server) and, indirectly, by the browser through HTTP.
In / out	In: HTTP requests. Out: JSON responses following the schemas.
Depended on by	Nothing imports main.py; it is the top of the backend import tree.

> **Observation (not a known intention):** `generate_response` and `LLMError` are imported **twice** in `main.py` (two separate import blocks). Python simply overwrites the name, so it is harmless, but it is redundant. The guide reports it as an observation rather than guessing why it happened.

app/schemas.py - the shape of the data

Aspect	Detail
Imports	`pydantic` BaseModel and Field.
Why it exists	FastAPI uses these classes to validate incoming JSON and to serialise outgoing JSON.

Aspect	Detail
Defines	**ChatRequest**, **ChatResponse**, **DocumentUploadResponse** (defined twice), **DocumentSearchRequest**, **RAGRequest**.
Called by	FastAPI itself when it parses a request body.
Depended on by	main.py imports these names.

> **Observation:** `DocumentUploadResponse` is declared **twice**. The first version has only three fields; the second (which wins, because it is defined later) adds `text_units`, `ocr_pages`, `tables`, and `images`. Only the second version is effective.

app/llm.py - talking to the language models

Aspect	Detail
Imports	`pathlib.Path`; `ollama.chat`.
Why it exists	It isolates all direct model calls so the rest of the code never talks to Ollama directly.
Defines	**LLMError**; **generate_response()** (text, **llama3.2**); **generate_multimodal_response()** (text + images, **qwen3.5:4b**).
Called by	`main.py` (chat and ask endpoints) and `test_multimodal_rag.py`.
Depended on by	main.py, and the test script.

app/context_engine/engine.py - context assembly

Aspect	Detail
Imports	`pathlib.Path`; `retrieve_context` from the RAG service.
Why it exists	It is meant to be the one authoritative place that decides what context is sent to the model.
Defines	**resolve_image_path()**; **assemble_context()**; **format_context()**.
Called by	main.py (`/documents/ask`) and test_multimodal_rag.py.
Calls	the RAG service, which in turn calls the embedding service and Pinecone.

app/rag/document_service.py - the processing orchestrator

Aspect	Detail
Imports	pathlib, uuid4, shutil, UploadFile, and every extractor plus the embedding and Pinecone services and the chunker.
Why it exists	It runs the whole PDF pipeline in the correct order and defines where files are stored.

Aspect	Detail
Defines	**save_document()**; **process_document()**; the constants **DOCUMENT_STORAGE_DIR** and **IMAGE_STORAGE_DIR**.
Called by	main.py (`/documents/upload` and `/documents/extract-preview`).

> **Important observation:** **process_document()** always calls **embed_content_units()** and **upsert_content_units()**. That means even the development endpoint `/documents/extract-preview` writes vectors into Pinecone as a side effect. There is no "dry run" mode.

app/rag/text_extractor.py - reading selectable text

Aspect	Detail
Imports	`fitz` (PyMuPDF).
Defines	**is_meaningful_text()**; **extract_text_from_pdf()**.
Calls	PyMuPDF page text extraction.
Called by	process_document().
Output	A list of text units and a list of page numbers that need OCR.

app/rag/ocr.py - reading text from page images

Aspect	Detail
Imports	io, os, pathlib, fitz, pytesseract, PIL.Image.
Defines	**TESSERACT_PATH** (a constant) and **extract_text_with_ocr()**.
Calls	PyMuPDF to render pages, Pillow to build an image, pytesseract to read it.
Called by	process_document() only for pages flagged by the text extractor.

app/rag/table_extractor.py - reading tables

Aspect	Detail
Imports	pathlib, `pdfplumber`.
Defines	**clean_cell()**; **table_to_searchable_text()**; **extract_tables_from_pdf()**.
Called by	process_document().
Output	A list of table units, each with headers, rows, and a searchable text form.

app/rag/image_extractor.py - pulling out pictures

Aspect	Detail
Imports	pathlib, fitz.

Aspect	Detail
Defines	**extract_images_from_pdf()**.
Calls	PyMuPDF to list and decode images, and writes files to disk.
Called by	process_document().
Output	A list of image units, each with a `content_ref` pointing at a saved file.

app/rag/chunker.py - making everything uniform

Aspect	Detail
Imports	`uuid4`.
Defines	**create_content_unit()**; **build_content_units()**.
Why it exists	Extractors each return slightly different dictionaries. The chunker reshapes them into one common structure so later stages only handle one shape.
Called by	process_document().

app/rag/embedding_service.py - turning text into vectors

Aspect	Detail
Imports	os, `dotenv.load_dotenv`, `google.genai`.
Defines	**generate_embedding()**; **generate_text_embedding()**; **get_embedding_text()**; **embed_content_units()**; the model name **gemini-embedding-001**.
Startup check	Raises RuntimeError at import time if GEMINI_API_KEY is missing.
Called by	process_document(), rag_service, main.py (`/documents/search`), and test scripts.

app/rag/pinecone_service.py - the vector database

Aspect	Detail
Imports	os, dotenv, `pinecone.Pinecone`.
Constants	**PINECONE_API_KEY**, **PINECONE_INDEX_NAME** (default `context-engineering-harness`), **PINECONE_NAMESPACE** = `multimodal-v2`.
Defines	**upsert_content_units()**; **search_similar_content()**.
Startup check	Raises RuntimeError at import time if PINECONE_API_KEY is missing.
Called by	process_document(), rag_service, main.py, and test scripts.

app/rag/rag_service.py - the retrieval glue

Aspect	Detail
Imports	`generate_text_embedding` and `search_similar_content`.
Defines	**retrieve_context()** - embed the query, then search.
Called by	`context_engine/engine.py`.

2.4 Frontend files, one by one

File	Responsibility	Depends on
index.html	The single HTML page. It has a `<div id='root'>` and loads `/src/main.jsx`.	browser
src/main.jsx	React entry point. Mounts the **App** component into `#root` using `createRoot`, wrapped in `StrictMode`. Imports `index.css`.	react, react-dom, App.jsx
src/App.jsx	The whole interface and all networking. Holds state, defines the fetch calls, and renders the upload and chat sections.	react (useState), App.css
src/App.css	Layout and appearance for the app: cards, buttons, chat bubbles, responsive rules.	none
src/index.css	Base template styles, colour variables, dark-mode variables, font stack.	none
package.json	Scripts (`dev`, `build`, `lint`, `preview`) and dependencies (react, react-dom, vite, oxlint).	npm
vite.config.js	Enables the React plugin for Vite.	vite
.oxlintrc.json	Rules for the Oxlint linter.	oxlint
README.md	The default template readme shipped with Vite; not project-specific.	none

2.5 Storage directories

Path	Holds	Naming
backend/storage/documents/	The original uploaded PDFs.	Each file is renamed to `<document_id>.pdf`.
backend/storage/images/	One sub-folder per document containing its extracted images.	`<document_id>/page_<n>_img_<k>.<ext>`

2.6 What does NOT exist

To avoid inventing structure, here is what the repository does **not** contain: no root-level README, no Dockerfile, no docker-compose, no CI configuration, no database, no authentication module, no

migration folder, no tests directory separate from the loose `test_*.py` scripts, and no knowledge-graph code.

Chapter 3. Complete System Architecture

This chapter builds the architecture picture piece by piece, then explains every arrow. Read the diagrams top to bottom: each one adds detail.

3.1 The big picture

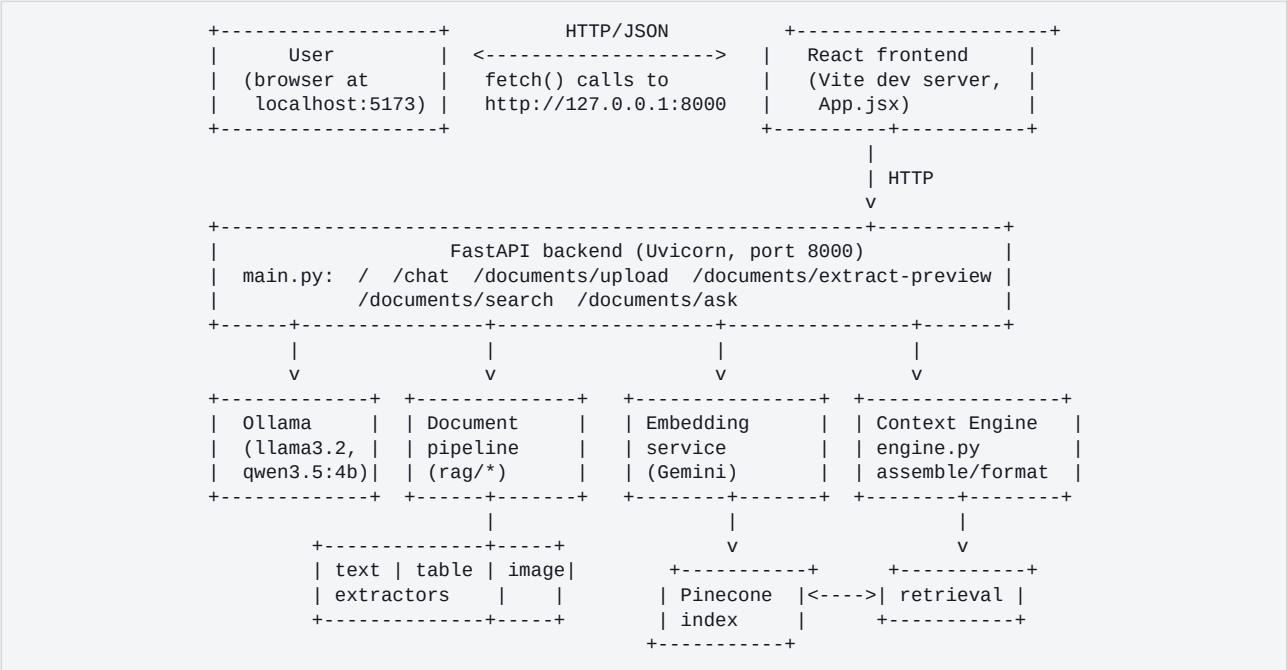


Figure 3.1 - top-level architecture

3.2 Arrow-by-arrow explanation

Arrow	What actually travels	Which code handles it
User → React	Keystrokes and a chosen file.	React state in `App.jsx`
React → FastAPI	JSON body or multipart form.	`fetch()` in `sendMessage` / `handleUpload`
FastAPI → schema layer	The raw JSON is parsed and validated.	Pydantic models in `schemas.py`
FastAPI → Ollama (/chat)	The user's message string.	`generate_response()` → `ollama.chat(model='llama3.2')`
FastAPI → document pipeline	The saved PDF path and its document id.	`process_document()`
Document pipeline → Gemini	Plain-text form of each content unit.	`embed_content_units()` → `generate_embedding()`
Document pipeline → Pinecone	Vector id + values + metadata.	`upsert_content_units()`

Arrow	What actually travels	Which code handles it
FastAPI → Context Engine (/documents/ask)	The query, top_k, and optional document id.	`assemble_context()`
Context Engine → RAG	The query text.	`retrieve_context()`
RAG → Gemini + Pinecone	Query text to a vector, then the vector to a search.	`generate_text_embedding()` + `search_similar_content()`
Context Engine → image resolution	A stored `content_ref` string.	`resolve_image_path()`
FastAPI → vision model	The prompt plus real image file paths.	`generate_multimodal_response()` → `ollama.chat(model='qwen3.5:4b')`

3.3 Upload and indexing architecture

```
React handleUpload()
      |  FormData { file:  }
      v
POST /documents/upload            (main.py: upload_document)
      |
      v
save_document(file)               -> writes storage/documents/.pdf
      |                               returns (document_id, filename, path)
      v
process_document(path, id)        (document_service.py)
      |
      +--> extract_text_from_pdf()   --> text_units + pages_needing_ocr
      |          |
      |          +--> extract_text_with_ocr()   (only flagged pages) -> more text_units
      |
      +--> extract_tables_from_pdf() --> table_units
      +--> extract_images_from_pdf() --> image_units (+ files on disk)
      |
      v
build_content_units(text, tables, images)   (chunker.py)
      |  -> one uniform list, sorted by (page_number, content_type)
      v
embed_content_units(units)         (embedding_service.py -> Gemini)
      |  -> each unit gains "embedding" (1024 numbers) and "embedding_text"
      v
upsert_content_units(embedded)     (pinecone_service.py)
      |  -> vectors {id, values, metadata} into namespace "multimodal-v2"
      v
DocumentUploadResponse  -> React shows the document id
```

Figure 3.2 - upload path

3.4 Retrieval and answer architecture (/documents/ask)

```
POST /documents/ask              (main.py: ask_document)
      |  RAGRequest {query, document_id, top_k}
      v
assemble_context(query, top_k, document_id)      (engine.py)
      |
      v
retrieve_context(...)            (rag_service.py)
      |
      +--> generate_text_embedding(query)  -> 1024-dim query vector
      +--> search_similar_content(vector, top_k, document_id)
                |
                v
           Pinecone index.query(namespace="multimodal-v2",
                                include_metadata=True,
                                filter={"document_id": {"$eq": id}} if id)
                |
                v
           list of results: id, score, content_type, page_number,
                            content, content_ref, source (+ table_number/image_id)
      |
      v
context = { "conversation": None, "memory": [],
            "rag": [ ...results... ], "knowledge_graph": [] }
      |
      v
format_context(context)  -> human-readable text with [TEXT/TABLE/IMAGE - Page n]
      |
      +--> for each image result: resolve_image_path(content_ref)
      |
      v
if any images: generate_multimodal_response(prompt, image_paths) -> qwen3.5:4b
else:          generate_response(prompt)                          -> llama3.2
      |
      v
JSON { query, answer, context, images_sent_to_llm }
```

Figure 3.3 - document question path

3.5 Where the vision model sits

There are two different language-model calls, and it matters which one is used when:

- **Text only - llama3.2.** Used by `/chat` and by `/documents/ask` when *no* image was retrieved.
- **Text + images - qwen3.5:4b.** Used by `/documents/ask` only when at least one retrieved result has `content_type == 'image'`. The file paths are passed to Ollama through the `images` field.

> **Important distinction:** Finding an image (retrieval) and understanding an image (vision) are two different steps. The image embedding itself is **not** visual - see Chapter 12. Only when the image's file path is handed to **qwen3.5:4b** does an actual model look at pixels.

Chapter 4. Application Startup and Execution

4.1 How the backend starts

The backend is a Python program. The project keeps its dependencies in a virtual environment at `backend/.venv`. The standard start command (run from inside the `backend` folder) is:

```
# activate the environment (Git Bash on Windows)
source .venv/Scripts/activate

# start the API server with auto-reload
uvicorn app.main:app --reload
```

Typical backend start commands

The string `app.main:app` means: “look in the Python module `app/main.py`, and find the object named `app`.” That object is created near the top of `main.py` with `app = FastAPI(...)`.

4.2 What Uvicorn does

- Uvicorn is a **web server**. It listens on a network port (by default 8000) and waits for HTTP requests.
- When a request arrives, Uvicorn does not run your function directly; it hands the request to the **ASGI application**, which is your FastAPI object.
- Because it is an ASGI server, it can handle asynchronous endpoints (note that `upload_document` and `extract_preview` are declared `async def`, while `chat` and others are plain `def`).
- `--reload` watches your files and restarts the server when they change, which is convenient during development.

4.3 How FastAPI creates the application

```
app = FastAPI(
    title="Context Engineering Harness",
    description="Backend API for the Context Engineering Harness.",
    version="1.0.0",
)
```

main.py - creating the app

This object collects every route you declare with a decorator such as `@app.post('/chat')`. FastAPI also builds an automatic interactive documentation page at `/docs` from the schemas.

4.4 How CORS is configured (and why)

CORS stands for *Cross-Origin Resource Sharing*. A browser treats the page served from `http://localhost:5173` (the React dev server) and the API at `http://127.0.0.1:8000` as two different *origins*, even though both are on your machine. By default the browser blocks JavaScript in one origin from reading responses from another. CORS headers tell the browser it is allowed.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

main.py - the CORS middleware

Only the two Vite dev-server origins are allowed. If you served the frontend from a different port or host, the browser would block the calls until that origin is added.

4.5 How imports connect the modules

When Uvicorn imports `app.main`, Python executes the file top to bottom. The import statements pull in other modules, and importing those modules runs *their* top-level code too. This is how the services are wired together - and it is also why some checks run at import time.

```
uvicorn imports app.main
      |
      +-- imports app.schemas            (defines request/response models)
      +-- imports app.llm                (defines generate_response, ...)
      +-- imports app.rag.document_service
      |        +-- imports app.rag.image_extractor     (needs fitz)
      |        +-- imports app.rag.text_extractor      (needs fitz)
      |        +-- imports app.rag.table_extractor     (needs pdfplumber)
      |        +-- imports app.rag.ocr                 (needs pytesseract, PIL)
      |        +-- imports app.rag.embedding_service   (loads .env, builds Gemini client)
      |        +-- imports app.rag.pinecone_service    (loads .env, builds Pinecone client)
      |        +-- imports app.rag.chunker
      +-- imports app.rag.embedding_service
      +-- imports app.rag.pinecone_service
      +-- imports app.context_engine.engine
               +-- imports app.rag.rag_service
```

Figure 4.1 - import graph at startup

> **Startup failure modes:** Because `embedding_service.py` and `pinecone_service.py` raise RuntimeError at import time when their keys are missing, a missing `GEMINI_API_KEY` or `PINECONE_API_KEY` prevents the server from starting at all.

4.6 How the React frontend starts

The frontend is started with npm scripts in the `frontend` folder:

```
npm install      # one time: install dependencies
npm run dev      # start the Vite dev server (default port 5173)
```

Frontend start commands

Vite serves `index.html`, which loads `src/main.jsx`. React then mounts **App** into the `#root` element. `npm run build` produces an optimised production bundle.

4.7 How the browser talks to FastAPI

All communication happens with the built-in `fetch()` function inside `App.jsx`. There is no API client library and no proxy: the URLs are written literally as `http://127.0.0.1:8000/...`.

4.8 What happens when the user sends a message

- React state `message` holds the text.
- `sendMessage()` checks it is not empty and not already loading.
- The user's line is appended to the `messages` array immediately (optimistic UI) and the input is cleared.
- `fetch('POST http://127.0.0.1:8000/chat', body = {"message": ...})` is sent.

- FastAPI validates the body against **ChatRequest**.
- `chat()` calls `generate_response()`, which calls Ollama's **llama3.2**.
- The reply is wrapped in **ChatResponse** and returned as JSON.
- React pushes the assistant line into `messages` and clears the loading flag.

If anything fails, the catch block appends an assistant message containing the error text, so the user sees a readable failure instead of silence.

4.9 What happens when the user uploads a PDF

- The file picker calls `handleFileChange()`, which rejects anything that is not a PDF.
- `handleUpload()` builds a **FormData** object and sends it to `/documents/upload`.
- FastAPI receives the file with `UploadFile = File(...)`.
- `save_document()` stores the PDF and returns a new `document_id`.
- `process_document()` runs the whole pipeline (extract, chunk, embed, index).
- The endpoint returns **DocumentUploadResponse** with counts.
- React stores the `document_id` and displays it.

> **Note on timing:** The upload endpoint is synchronous and does all its work before responding. Uploading a large PDF (the storage folder shows files of over 100 MB have been tested) means the browser waits until the entire pipeline finishes; the button simply shows “Processing...”. There is no progress reporting and no background job.

Chapter 5. Complete Code Walkthrough

This chapter walks the important functions in the order they are used. For each one you get a plain-language explanation of the problem it solves, its inputs and outputs, who calls it, and what happens on success and failure. Short excerpts are shown, not whole files, so the point stays readable.

5.1 schemas.py - defining the shape of data

A **schema** is a description of what a piece of data must look like. FastAPI uses these classes to check incoming JSON and to build outgoing JSON.

ChatRequest

```
        class ChatRequest(BaseModel):
            message: str = Field(min_length=1,
                                 description="Message sent by the user")
```

ChatResponse

```
        class ChatResponse(BaseModel):
            response: str
```

It defines the single field the chat endpoint returns: `response`. Its existence means the API response is predictable.

DocumentUploadResponse (the effective one)

```
        class DocumentUploadResponse(BaseModel):
            document_id: str
            filename: str
            status: str
            text_units: int
            ocr_pages: int
            tables: int
            images: int
```

This is the **second** definition in the file, so it is the one that counts. It describes the statistics returned after an upload: the id, the original name, a status word, and four counts.

DocumentSearchRequest and RAGRequest

```
        class DocumentSearchRequest(BaseModel):
            query: str = Field(min_length=1)
            document_id: str | None = None
            top_k: int = Field(default=5, ge=1, le=20)

        class RAGRequest(BaseModel):
            query: str = Field(min_length=1)
            document_id: str | None = None
            top_k: int = Field(default=5, ge=1, le=20)
```

5.2 llm.py - calling the language models

LLMError

A tiny custom exception class. Its purpose is to give callers a single, predictable error type for “the model failed” so they do not have to catch every possible Ollama or network exception individually. The endpoints catch `LLMError` and translate it into HTTP 503.

generate_response(message)

```
        def generate_response(message: str) -> str:
            try:
                response = chat(
                    model="llama3.2",
                    messages=[{"role": "user", "content": message}],
                )
                return response.message.content
            except Exception as error:
                print("LLM ERROR:", repr(error))
                raise LLMError(
                    f"The language model could not generate a response: {error}"
                ) from error
```

generate_multimodal_response(message, image_paths)

```
        def generate_multimodal_response(message: str, image_paths: list[str]) -> str:
            try:
                valid_images = []
                for image_path in image_paths:
                    path = Path(image_path)
                    if path.exists():
                        valid_images.append(str(path))
                if not valid_images:
                    raise ValueError("No valid image files were found.")
                response = chat(
                    model="qwen3.5:4b",
                    messages=[{"role": "user", "content": message,
                               "images": valid_images}],
                )
                return response.message.content
            except Exception as error:
                raise LLMError(f"Multimodal model error: {error}") from error
```

5.3 main.py - the endpoints

The application object and middleware

At import time FastAPI is constructed, CORS is added, and every decorated function is registered as a route. Nothing here calls the models yet; the work happens when a request arrives.

root() - GET /

```
        @app.get("/")
        def root():
            return {"message": "Context Engineering Harness API is running."}
```

A health check. It proves the server is up and returns a fixed JSON object. It calls nothing else.

chat(request) - POST /chat

```
        @app.post("/chat", response_model=ChatResponse)
        def chat(request: ChatRequest):
            try:
                response = generate_response(request.message)
            except LLMError as error:
                raise HTTPException(status_code=503, detail=str(error))
            return ChatResponse(response=response)
```

upload_document(file) - POST /documents/upload

```
@app.post("/documents/upload", response_model=DocumentUploadResponse)
async def upload_document(file: UploadFile = File(...)):
    try:
        (document_id, original_filename, document_path) = save_document(file)
        (content_units, ocr_pages, table_units, image_units) = process_document(
            document_path=document_path, document_id=document_id)
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error))

    text_unit_count = sum(1 for unit in content_units
                          if unit["content_type"] == "text")
    return DocumentUploadResponse(
        document_id=document_id, filename=original_filename,
        status="processed", text_units=text_unit_count,
        ocr_pages=len(ocr_pages), tables=len(table_units),
        images=len(image_units))
```

extract_preview(file) - POST /documents/extract-preview

A development helper. It calls exactly the same two functions as the upload endpoint, then returns the **entire** extraction result - every content unit, the OCR page list, the tables and the images - instead of just counts.

> **Side effect to be aware of:** Because it calls **process_document()**, this “preview” endpoint also generates embeddings and writes vectors to Pinecone. It is not a read-only preview.

search_documents(request) - POST /documents/search

```
@app.post("/documents/search")
def search_documents(request: DocumentSearchRequest):
    try:
        query_embedding = generate_embedding(request.query)
        results = search_similar_content(
            query_embedding=query_embedding,
            top_k=request.top_k, document_id=request.document_id)
    except ValueError as error:
        raise HTTPException(status_code=400, detail=str(error))
    return {"query": request.query, "results": results}
```

ask_document(request) - POST /documents/ask

This is the full RAG endpoint. It is worth reading in four parts.

```
# part 1 - assemble and format context
context = assemble_context(query=request.query, top_k=request.top_k,
                           document_id=request.document_id)
formatted_context = format_context(context)

# part 2 - collect image paths
image_paths = []
for result in context["rag"]:
    if result["content_type"] != "image":
        continue
    image_paths.append(resolve_image_path(result["content_ref"]))

# part 3 - build the prompt
prompt = f"""...instructions... Retrieved Context: {formatted_context}
User Question: {request.query}"""

# part 4 - choose the model
if image_paths:
    answer = generate_multimodal_response(message=prompt, image_paths=image_paths)
else:
    answer = generate_response(prompt)
```

5.4 document_service.py - the orchestrator

save_document(file)

```
def save_document(file: UploadFile) -> tuple[str, str, Path]:
    DOCUMENT_STORAGE_DIR.mkdir(parents=True, exist_ok=True)
    original_filename = file.filename or "unknown.pdf"
    extension = Path(original_filename).suffix.lower()
    if extension != ".pdf":
        raise ValueError("Only PDF documents are supported.")
    document_id = str(uuid4())
    destination = DOCUMENT_STORAGE_DIR / f"{document_id}.pdf"
    with destination.open("wb") as buffer:
        shutil.copyfileobj(file.file, buffer)
    return (document_id, original_filename, destination)
```

process_document(document_path, document_id)

```
text_units, pages_needing_ocr = extract_text_from_pdf(...)
if pages_needing_ocr:
    text_units.extend(extract_text_with_ocr(..., pages=pages_needing_ocr))
table_units = extract_tables_from_pdf(...)
image_units = extract_images_from_pdf(..., image_storage_dir=IMAGE_STORAGE_DIR)
content_units = build_content_units(text_units, table_units, image_units)
embedded_units = embed_content_units(content_units)
upsert_content_units(embedded_units)
return (content_units, pages_needing_ocr, table_units, image_units)
```

5.5 text_extractor.py - selectable text

is_meaningful_text(text)

```
def is_meaningful_text(text: str) -> bool:
    cleaned_text = text.strip()
    if len(cleaned_text) < 20:
        return False
    alphanumeric_count = sum(c.isalnum() for c in cleaned_text)
    return alphanumeric_count >= 10
```

extract_text_from_pdf(document_path, document_id)

It opens the PDF with PyMuPDF, loops over every page with an index, and converts the zero-based index to a human page number (`page_index + 1`). For each page it calls `page.get_text('text')`. Good pages become text units; poor pages are appended to `pages_needing_ocr`. The file is closed in a `finally` block so the handle is never leaked. It returns both lists.

```
text_unit = {
    "document_id": document_id,
    "content_type": "text",
    "page_number": page_number,
    "content": text,
    "source": "pdf_text",
}
```

The shape of a text unit from the PDF

5.6 ocr.py - reading page images

TESSERACT_PATH and the pytesseract setting

```
        TESSERACT_PATH = os.getenv(
            "TESSERACT_PATH",
            r"C:\Program Files\Tesseract-OCR\tesseract.exe")
        pytesseract.pytesseract.tesseract_cmd = TESSERACT_PATH
```

This tells pytesseract where the Tesseract program lives. On this project the default value is the standard Windows install path; it can be overridden by the `TESSERACT_PATH` environment variable. This variable is **not** in the current `.env`, so the default is used.

extract_text_with_ocr(document_path, document_id, pages)

```
        for page_number in pages:
            page = pdf_document[page_number - 1]
            pixmap = page.get_pixmap(matrix=fitz.Matrix(2, 2), alpha=False)
            image = Image.open(io.BytesIO(pixmap.tobytes("png")))
            text = pytesseract.image_to_string(image).strip()
            if not text:
                continue
            text_units.append({
                "document_id": document_id, "content_type": "text",
                "page_number": page_number, "content": text, "source": "ocr"})
```

5.7 table_extractor.py - structured tables

clean_cell(cell)

Turns a possibly-empty cell into a tidy string: None becomes an empty string, and any run of whitespace collapses into single spaces. This makes later comparisons and embeddings tidier.

table_to_searchable_text(headers, rows)

```
        for row in rows:
            values = []
            for index, header in enumerate(headers):
                value = row[index] if index < len(row) else ""
                values.append(f"{header}: {value}")
            searchable_rows.append(" | ".join(values))
        return "\n".join(searchable_rows)
```

This is the key idea of the table module. Instead of asking the embedding model to understand a grid, it rewrites each row as a sentence of `header: value` pairs joined by `|`. The column name is repeated in every row, which helps the model keep the meaning when it embeds the text.

extract_tables_from_pdf(document_path, document_id)

It opens the PDF with pdfplumber, iterates pages with `start=1` so page numbers are human-friendly, and calls `page.extract_tables()`. For each table it drops empty rows, cleans every cell, treats the first row as headers and the rest as data, then builds the searchable text and a table unit. A document-level counter numbers the tables across the whole file.

> **Limitation:** Header detection is simply “the first row”. Tables without a header row, multi-row headers, or merged cells are not specially handled.

5.8 image_extractor.py - saving pictures

extract_images_from_pdf(document_path, document_id, image_storage_dir)

```
        document_image_dir = image_storage_dir / document_id
        document_image_dir.mkdir(parents=True, exist_ok=True)
        for page_number, page in enumerate(pdf_document, start=1):
            for image_index, image_info in enumerate(page.get_images(full=True), start=1):
                xref = image_info[0]
                image_data = pdf_document.extract_image(xref)
                image_filename = f"page_{page_number}_img_{image_index}.{image_data['ext']}"
                image_path = document_image_dir / image_filename
                image_path.write_bytes(image_data["image"])
                content_ref = str(image_path.relative_to(
                    image_storage_dir.parent)).replace("\\", "/")
```

5.9 chunker.py - one shape for everything

create_content_unit(...)

```
        def create_content_unit(document_id, content_type, page_number,
                                content="", content_ref=None, metadata=None) -> dict:
            return {
                "id": str(uuid4()),
                "document_id": document_id,
                "content_type": content_type,
                "page_number": page_number,
                "content": content,
                "content_ref": content_ref,
                "metadata": metadata or {},
            }
```

Every unit of information - a block of text, a table, or an image - becomes this same dictionary. The `id` is the future Pinecone vector id. `content` holds text for text/table units (empty for images) and `content_ref` points at a file for images. `metadata` carries type-specific extras.

build_content_units(text_units, table_units, image_units)

It loops the three input lists, converts each item with `create_content_unit`, and produces one combined list. Text units store a `source`; table units store source, table_number, headers and rows; image units store source, image_id and image_format. Finally it sorts everything:

```
        content_units.sort(key=lambda unit: (unit["page_number"], unit["content_type"]))
```

> **Observation:** Sorting by `content_type` is alphabetical, so on the same page the order is **image**, then **table**, then **text**. This restores page order but not necessarily the original reading order within a page.

5.10 embedding_service.py - text becomes numbers

Startup behaviour

```
        load_dotenv()
        GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
        if not GEMINI_API_KEY:
            raise RuntimeError("GEMINI_API_KEY is not configured.")
        client = genai.Client(api_key=GEMINI_API_KEY)
        EMBEDDING_MODEL = "gemini-embedding-001"
```

This runs as soon as the module is imported. The model name is `gemini-embedding-001`, stored in a constant so it is easy to see.

generate_embedding(text)

```
        def generate_embedding(text: str) -> list[float]:
            if not text.strip():
                raise ValueError("Cannot generate an embedding for empty text.")
            response = client.models.embed_content(
                model=EMBEDDING_MODEL,
                contents=text,
                config=types.EmbedContentConfig(output_dimensionality=1024),
            )
            return response.embeddings[0].values
```

generate_text_embedding(text)

A thin wrapper that simply calls `generate_embedding`. Its name documents intent: this is the embedding used for RAG queries. It returns the same 1024-number list.

get_embedding_text(content_unit)

```
        if content_type == "text":
            return content_unit["content"]
        if content_type == "table":
            return content_unit["content"]
        if content_type == "image":
            return f"Image {image_id} extracted from page {page_number} of the document."
        raise ValueError(f"Unsupported content type: {content_type}")
```

> **Crucial to understand:** For an image, the text that is embedded is a **sentence describing the image's metadata**, not the image itself. This is not a visual embedding. See Chapter 12 for the full explanation.

embed_content_units(content_units)

It loops over every unit, decides its embedding text with `get_embedding_text`, calls `generate_embedding`, and returns a new list where each unit has two extra keys: `embedding` (the vector) and `embedding_text` (what was embedded, useful for debugging). The original keys are kept.

5.11 pinecone_service.py - the vector database

Startup behaviour and constants

```
        PINECONE_API_KEY = os.getenv("PINECONE_API_KEY")
        PINECONE_INDEX_NAME = os.getenv("PINECONE_INDEX_NAME", "context-engineering-harness")
        PINECONE_NAMESPACE = "multimodal-v2"
        if not PINECONE_API_KEY:
            raise RuntimeError("PINECONE_API_KEY is not configured.")
        pinecone_client = Pinecone(api_key=PINECONE_API_KEY)
        index = pinecone_client.Index(PINECONE_INDEX_NAME)
```

Note the two things you can see here: the index name can be overridden by `PINECONE_INDEX_NAME` but defaults to `context-engineering-harness`, and the namespace is hard-coded to `multimodal-v2`. The comment in the code explains that the namespace “keeps new multimodal vectors isolated from old vectors”.

upsert_content_units(embedded_units)

```
        for unit in embedded_units:
            metadata = {
                "document_id": unit["document_id"],
                "content_type": unit["content_type"],
                "page_number": unit["page_number"],
                "content": unit.get("content", ""),
                "content_ref": unit.get("content_ref") or "",
                "source": unit["metadata"].get("source", "unknown"),
            }
            if unit["content_type"] == "table":
                metadata["table_number"] = unit["metadata"].get("table_number", 0)
            if unit["content_type"] == "image":
                metadata["image_id"] = unit["metadata"].get("image_id", "")
            vectors.append({"id": unit["id"], "values": unit["embedding"],
                            "metadata": metadata})
        if not vectors:
            return 0
        index.upsert(vectors=vectors, namespace=PINECONE_NAMESPACE)
        return len(vectors)
```

search_similar_content(query_embedding, top_k, document_id)

```
        search_filter = None
        if document_id:
            search_filter = {"document_id": {"$eq": document_id}}
        response = index.query(
            vector=query_embedding, top_k=top_k,
            include_metadata=True, namespace=PINECONE_NAMESPACE,
            filter=search_filter)
        for match in response.matches:
            ... build result dict from match.metadata ...
            if "table_number" in metadata: result["table_number"] = ...
            if "image_id" in metadata:     result["image_id"] = ...
```

5.12 rag_service.py - the retrieval glue

```
        def retrieve_context(query, top_k=5, document_id=None):
            query_embedding = generate_text_embedding(query)
            results = search_similar_content(
                query_embedding=query_embedding, top_k=top_k, document_id=document_id)
            return results
```

Two steps in one place: embed the query, then search. Keeping this in its own function means the Context Engine never has to know how retrieval is implemented. It is called by `assemble_context`.

5.13 engine.py - the Context Engine

resolve_image_path(content_ref)

```
        if not content_ref:
            raise ValueError("Image content reference is empty.")
        path = Path(content_ref)
        if path.is_absolute():
            resolved_path = path
        else:
            resolved_path = BASE_DIR / "storage" / path
        resolved_path = resolved_path.resolve()
        if not resolved_path.exists():
            raise FileNotFoundError(f"Image file not found: {resolved_path}")
        return str(resolved_path)
```

assemble_context(query, top_k, document_id)

```
        rag_results = retrieve_context(query=query, top_k=top_k, document_id=document_id)
        context = {
            "conversation": None,
            "memory": [],
            "rag": rag_results,
            "knowledge_graph": [],
        }
        return context
```

This is the heart of the Context Engine as it stands today. It performs retrieval and places the results into a four-part dictionary. Only the `rag` slot is populated with real data; the other three are fixed empty placeholders.

> **Status:** `conversation` is always **None**, `memory` is always an empty list, and `knowledge_graph` is always an empty list. These are reserved slots for future work, not working features.

format_context(context)

```
        for result in context["rag"]:
            content_type = result["content_type"]
            page_number = result["page_number"]
            if content_type == "text":
                sections.append(f"[TEXT - Page {page_number}]\n{result['content']}")
            elif content_type == "table":
                sections.append(f"[TABLE - Page {page_number}]\n{result['content']}")
            elif content_type == "image":
                sections.append(f"[IMAGE - Page {page_number}]\n"
                                f"Image reference: {result['content_ref']}")
        return "\n\n".join(sections)
```

5.14 App.jsx - the frontend

State

```
        const [message, setMessage] = useState("");
        const [messages, setMessages] = useState([]);
        const [loading, setLoading] = useState(false);
        const [selectedFile, setSelectedFile] = useState(null);
        const [documentId, setDocumentId] = useState("");
        const [uploading, setUploading] = useState(false);
        const [uploadError, setUploadError] = useState("");
```

Seven pieces of state drive the whole interface. `messages` is the conversation list; `message` is the controlled textarea value; `loading` and `uploading` drive disabled buttons and the “Thinking...” bubble.

sendMessage()

Guards against empty or duplicate sends, appends the user line optimistically, clears the box, sets loading, sends a JSON POST to `/chat`, and then either appends the assistant reply or appends an error message. The `finally` block always clears loading.

handleKeyDown(event)

Enter sends the message; Shift+Enter inserts a newline. This is achieved by checking `event.key === 'Enter'` and `!event.shiftKey`, then calling `preventDefault()` so the newline is not added.

handleFileChange(event)

Takes the first chosen file, clears old errors and any previous document id, and rejects anything whose MIME type is not `application/pdf` and whose name does not end in `.pdf`. This mirrors (but does not replace) the server-side check.

handleUpload()

Requires a selected file, sets uploading, builds a **FormData** and appends the file under the field name `file` (which matches FastAPI's `File(...)`), posts to `/documents/upload`, and on success stores `data.document_id`. Errors become text in `uploadError`, and the `finally` block clears the uploading state.

Rendering

The JSX returns a single `<div className='app'>` with two sections: an **upload section** (file input, selected-name text, upload button, error text, and the document id block) and a **chat section** (scrollable message list, a transient “Thinking...” bubble while loading, a textarea, and a Send button). Messages are rendered by mapping the `messages` array; the CSS class depends on the `role` so user and assistant bubbles align differently.

Chapter 6. Function Calling Map: Who Calls Whom

This chapter answers one question: when you click something, which functions run, in what order, and what data passes between them? The names below are the real names in the code.

6.1 Chat request call graph

```
[User clicks Send]
      |
      v
App.jsx : sendMessage()
      |  fetch("POST http://127.0.0.1:8000/chat", {message})
      v
main.py : chat(request: ChatRequest)      <-- FastAPI validates via ChatRequest
      |
      v
llm.py  : generate_response(message)
      |
      v
ollama.chat(model="llama3.2", messages=[...])
      |
      v
ChatResponse(response="...")   ->  JSON  { "response": "..." }
      |
      v
App.jsx : setMessages([... , {role:"assistant", content: data.response}])
      |
      v
[Bubble appears in the chat box]
```

Figure 6.1 - plain chat call graph

6.2 PDF upload call graph

```
[User clicks Upload PDF]
      |
      v
App.jsx : handleUpload()
      |  FormData { file }
      |  fetch("POST /documents/upload", formData)
      v
main.py : upload_document(file: UploadFile)
      |
      +--> document_service.save_document(file)
      |         -> storage/documents/.pdf
      |         returns (document_id, filename, path)
      |
      +--> document_service.process_document(path, document_id)
               |
               +--> text_extractor.extract_text_from_pdf(path, id)
               |         -> (text_units, pages_needing_ocr)
               |
               +--> ocr.extract_text_with_ocr(path, id, pages_needing_ocr)   [only if any]
               |         -> ocr text_units (appended to text_units)
               |
               +--> table_extractor.extract_tables_from_pdf(path, id)
               |         -> table_units
               |
               +--> image_extractor.extract_images_from_pdf(path, id, IMAGE_STORAGE_DIR)
               |         -> image_units (+ files written under storage/images//)
               |
               +--> chunker.build_content_units(text_units, table_units, image_units)
               |         -> content_units (sorted by page, then type)
               |
               +--> embedding_service.embed_content_units(content_units)
               |         -> embedded_units (each with "embedding", "embedding_text")
               |
               +--> pinecone_service.upsert_content_units(embedded_units)
                         -> index.upsert(namespace="multimodal-v2")
      |
      v
DocumentUploadResponse(document_id, filename, "processed", counts...)
      |
      v
App.jsx : setDocumentId(data.document_id)  +  [shows the id]
```

Figure 6.2 - upload call graph

6.3 Document question call graph

```
[Client sends POST /documents/ask]  (no UI button yet)
      |
      v
main.py : ask_document(request: RAGRequest)
      |
      +--> context_engine.engine.assemble_context(query, top_k, document_id)
      |         |
      |         +--> rag_service.retrieve_context(query, top_k, document_id)
      |                  |
      |                  +--> embedding_service.generate_text_embedding(query)
      |                  |         -> generate_embedding(query) -> Gemini
      |                  |         -> 1024-dim query vector
      |                  |
      |                  +--> pinecone_service.search_similar_content(
      |                          vector, top_k, doc_id)
      |                            -> index.query(namespace="multimodal-v2",
      |                                           filter=...)
      |                            -> list of result dicts
      |         returns { "conversation": None, "memory": [],
      |                   "rag": [...], "knowledge_graph": [] }
      |
      +--> context_engine.engine.format_context(context)
      |         -> prompt-ready text string
      |
      +--> for each image result: engine.resolve_image_path(content_ref)
      |         -> absolute filesystem path
      |
      +--> llm.generate_multimodal_response(prompt, image_paths)   [if images]
      |    OR  llm.generate_response(prompt)                        [if no images]
      |         -> ollama.chat(model="qwen3.5:4b" or "llama3.2")
      |
      v
JSON { query, answer, context, images_sent_to_llm }
```

Figure 6.3 - document question call graph

6.4 Semantic search call graph

```
POST /documents/search
      |
      v
main.py : search_documents(request: DocumentSearchRequest)
      |
      +--> embedding_service.generate_embedding(request.query)
      +--> pinecone_service.search_similar_content(query_embedding=..., ...)
      |
      v
{ "query": ..., "results": [ ... ] }
```

Figure 6.4 - search call graph

6.5 A flat direct/indirect map

Function	Called directly by	Calls (indirectly)
generate_response	main.chat, main.ask_document	ollama.chat (llama3.2)
generate_multimodal_response	main.ask_document, test_multimodal_rag	ollama.chat (qwen3.5:4b)
save_document	main.upload_document, main.extract_preview	shutil.copyfileobj, uuid4
process_document	main.upload_document, main.extract_preview	all extractors, chunker, embedding_service, pinecone_service
extract_text_from_pdf	process_document	fitz page.get_text

Function	Called directly by	Calls (indirectly)
extract_text_with_ocr	process_document	fitz get_pixmap, PIL, pytesseract
extract_tables_from_pdf	process_document	pdfplumber page.extract_tables
extract_images_from_pdf	process_document	fitz get_images/extract_image, Path.write_bytes
build_content_units	process_document	create_content_unit, list.sort
embed_content_units	process_document	get_embedding_text, generate_embedding
upsert_content_units	process_document, test_indexing	index.upsert
search_similar_content	main.search_documents, rag_service, test scripts	index.query
retrieve_context	assemble_context	generate_text_embedding, search_similar_content
assemble_context	main.ask_document, test_multimodal_rag	retrieve_context
format_context	main.ask_document	nothing (pure formatting)
resolve_image_path	main.ask_document, test_multimodal_rag	Path.exists

Chapter 7. Chat Pipeline Explained

This chapter follows the plain chat feature from a single typed message to the answer on screen. It is deliberately separate from document question answering.

7.1 The React state behind the chat

Three state variables matter here:

- `message` - the text currently in the box (a **controlled input**: the textarea's value comes from state and every keystroke updates it).
- `messages` - an array of `{ role, content }` objects that is the conversation.
- `loading` - a boolean that disables the button and shows “Thinking...” in the reply bubble and “Sending...” on the button.

7.2 How the request is built and sent

```
const response = await fetch("http://127.0.0.1:8000/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message: userMessage }),
});
const data = await response.json();
```

JSON (JavaScript Object Notation) is a text format for structured data. `JSON.stringify` turns the JavaScript object into that text. The header `Content-Type: application/json` tells the server how to read the body. For a normal chat there is no `document_id`; the endpoint does not accept one.

7.3 How the backend validates the input

FastAPI reads the raw JSON and builds a **ChatRequest**. Pydantic checks that `message` exists and is at least one character. If the check fails, FastAPI answers with HTTP 422 and `chat()` never runs - the model is never called with invalid input.

7.4 How the endpoint calls the model

`chat()` wraps the call in a try/except. On success it returns `ChatResponse(response=...)`. On an **LLMError** it raises `HTTPException(503, str(error))`. FastAPI serialises the response to JSON using the schema.

7.5 A complete worked example

Stage	Data
User types	What medicine was prescribed?
React state	`message = "What medicine was prescribed?"`
Request body	`{"message": "What medicine was prescribed?"}`
Validated object	`ChatRequest(message="What medicine was prescribed?")`
LLM call	`generate_response("What medicine was prescribed?")` → Ollama llama3.2

Stage	Data
Model reply	“I don't have enough context to know which medicine was prescribed.”
HTTP response	`{"response": "I don't have enough context..."}`
React appends	`{ role: "assistant", content: "I don't have enough context..." }`
Screen	An assistant bubble with that text; loading turned off.

> **Why the model does not know the answer:** The plain chat endpoint sends only your text. It does **not** retrieve anything from your documents. If you want document content to be used, that is the separate `/documents/ask` path - and the interface does not call it yet.

7.6 Loading and error states

- **Loading on:** the moment `fetch` is called, before the await resolves.
- **Loading off:** in the `finally` block, so it clears on success *and* failure.
- **HTTP error path:** if `response.ok` is false, the code throws with the server's `detail` text; the catch block appends it as an assistant message.
- **Network error path:** if the backend is not running, fetch itself rejects and the catch shows a connection message.

7.7 What happens when the LLM fails

```
ollama unavailable / model missing
      |
      v
generate_response raises LLMError
      |
      v
chat() catches LLMError -> HTTPException(503, "...")
      |
      v
FastAPI returns { "detail": "The language model could not generate a response: ..." }
      |
      v
React !response.ok -> throw new Error(data.detail)
      |
      v
catch -> assistant bubble shows the detail text
```

Figure 7.1 - failure path for a chat request

Chapter 8. PDF Upload and Document Processing

8.1 The complete lifecycle of one PDF

```
PDF chosen in browser
      |
      v  (1) frontend checks the extension / MIME type
handleUpload()  ->  FormData  ->  POST /documents/upload
      |
      v  (2) server checks again and creates a document id
save_document()  ->  storage/documents/.pdf
      |
      v  (3) text extraction
extract_text_from_pdf()  ->  good pages become text units
                             bad pages are listed for OCR
      |
      v  (4) OCR fallback (only for the bad pages)
extract_text_with_ocr()  ->  more text units (source = "ocr")
      |
      v  (5) tables
extract_tables_from_pdf()  ->  table units (headers, rows, searchable text)
      |
      v  (6) images
extract_images_from_pdf()  ->  image files on disk + image units (content_ref)
      |
      v  (7) unification
build_content_units()  ->  one list of uniform content units, sorted by page
      |
      v  (8) embeddings
embed_content_units()  ->  each unit gains a 1024-number vector
      |
      v  (9) indexing
upsert_content_units()  ->  vectors + metadata stored in Pinecone
```

Figure 8.1 - the nine stages after an upload

8.2 Upload validation (two layers)

Validation happens in two places, on purpose. The browser check gives fast feedback; the server check is the one that actually protects the system, because a browser check can be bypassed.

Layer	Where	What it checks
Client	App.jsx handleFileChange	MIME type is application/pdf OR name ends with .pdf
Server	document_service.save_document	Path(name).suffix.lower() == '.pdf'

8.3 Document IDs

A **document id** is a unique label created for each upload. The code uses a UUID (`uuid4()`) - a random string like `0b3ccb21-91eb-441d-af84-ee9403407643`. Why a random id instead of the original filename? Because two users could upload files with the same name, and because the original name might contain characters that are unsafe as a filename. The original name is kept only for display.

The id is the thread that ties everything together: it names the stored PDF, it is copied into every content unit and every Pinecone metadata record, and it is returned to the browser so a later search or question can be restricted to one document.

8.4 The difference between the six artefacts

This table is important because beginners often confuse these things. They are six different things that all relate to one original document.

Artefact	What it is	Where it lives
Original PDF	The file the user uploaded, byte for byte.	storage/documents/<id>.pdf
Extracted text	Characters read from the PDF's text layer for pages that have one.	In memory, then in content units (source='pdf_text')
OCR output	Characters read from a *picture* of a page by Tesseract.	In memory, then in content units (source='ocr')
Table's structured form	The headers list and the list of rows, plus a searchable text version.	In memory, in content units (and partly in Pinecone metadata)
Image file	An actual picture written to disk (.png/.jpg/.jpeg/...).	storage/images/<id>/page_n_img_k.ext
Content unit	A uniform dictionary describing one piece of content.	In memory during processing (and mirrored, partly, in Pinecone metadata)
Embedding	A list of 1024 numbers summarising the unit's text.	Attached to the unit in memory, then stored as the vector values in Pinecone
Pinecone record	One vector: an id, 1024 values, and a metadata dictionary.	Pinecone, namespace 'multimodal-v2'

8.5 One page producing several content units

Imagine a page that has a short heading, a paragraph, a table, and a diagram image.

```
Page 3 of report.pdf
+--------------------------------------------------+
|  "Lab Results - March 2026"      (selectable text)|
|  "The patient showed improvement..." (text)       |
|  +----------------+----------------+              |
|  | Test           | Result         |  (a table)   |
|  | Glucose        | 95 mg/dL       |              |
|  +----------------+----------------+              |
|  [ a small bar chart image ]                     |
+--------------------------------------------------+

After extraction on page 3, the units are (sorted by page, then type):
  1. image  unit  (page 3, content_ref="images//page_3_img_1.png")
  2. table  unit  (page 3, headers=["Test","Result"],
                   rows=[["Glucose","95 mg/dL"]],
                   content="Test: Glucose | Result: 95 mg/dL")
  3. text   unit  (page 3, content="Lab Results - March 2026\n\n"
                                   "The patient showed improvement...")
```

Figure 8.2 - one page, three content units

Why text is one unit here, not several: Although the module is called **chunker**, `build_content_units` creates **one text unit per page** - it does not split a long page into paragraphs. So “chunking” in this project currently means *page-level units*, not sentence-level chunks.

8.6 How ids and page numbers connect content to its source

Every unit carries `document_id` and `page_number`. Together with `content_type` they answer the retrieval-time question “where did this text come from?” A retrieved table can be reported as “Table on page 3 of document 0b3c...”. For images, the extra `content_ref` lets the program turn the reference back into an absolute path on disk, so the actual picture can be handed to a vision model.

Chapter 9. OCR Explained

9.1 What OCR means

OCR stands for *Optical Character Recognition*. It is software that looks at a picture containing text and figures out which letters and numbers it contains. A simple example: you photograph a printed receipt, and OCR turns those pixels into the words “Total 12.50”.

9.2 Why ordinary PDF text extraction sometimes fails

A PDF can store text in two very different ways:

Type	What it contains	Can PyMuPDF read the text directly?
Digital (born-digital) PDF	Real text characters with fonts and positions.	Yes - get_text returns the words.
Scanned PDF	A big picture of a page; the 'text' is only pixels.	No - get_text returns little or nothing.

If a document is simply photos of paper, there is no text layer to read, so OCR is the only way to recover the words.

9.3 How the code decides which pages need OCR

This decision is made by `is_meaningful_text()` inside `text_extractor.py`. After stripping whitespace, a page's text must be at least 20 characters long and contain at least 10 letters or digits. If it fails, the page number is added to `pages_needing_ocr`. Only those pages are sent onward, so OCR cost is paid where it is needed.

9.4 Rendering a page as an image

```
        pixmap = page.get_pixmap(matrix=fitz.Matrix(2, 2), alpha=False)
        image_bytes = pixmap.tobytes("png")
        image = Image.open(io.BytesIO(image_bytes))
```

ocr.py - page to image

PyMuPDF renders the page into a raster image (a grid of coloured pixels). `Matrix(2, 2)` doubles the width and height, producing a crisper picture, because OCR accuracy usually improves when text is larger. `alpha=False` means no transparency channel - Tesseract works on ordinary RGB images. The bytes are loaded into a Pillow image, which is the format pytesseract expects.

9.5 What Tesseract does

Tesseract is an open-source OCR engine. `pytesseract.image_to_string(image)` sends the image to the Tesseract program and returns the text it recognised. The location of that program is set with `pytesseract.pytesseract.tesseract_cmd = TESSERACT_PATH`.

9.6 How OCR output becomes a text unit and continues downstream

```
        text = pytesseract.image_to_string(image).strip()
        if not text:
            continue
        text_unit = {
            "document_id": document_id,
            "content_type": "text",
            "page_number": page_number,
            "content": text,
            "source": "ocr",
        }
```

Notice the shape is identical to a normal text unit - the only difference is `source` is `'ocr'` instead of `'pdf_text'`. That is deliberate: downstream code (chunker, embedding, Pinecone) does not need to care where the words came from. The `source` value is preserved in metadata so a result can later be labelled as OCR-derived.

9.7 Limitations of the current OCR implementation

- It assumes Tesseract is installed, and by default at the Windows path `C:\Program Files\Tesseract-OCR\tesseract.exe` unless `TESSERACT_PATH` is set.
- It uses Tesseract's default language configuration - no explicit language is passed.
- It does no image pre-processing (no de-skew, denoise, or binarisation), so poor scans will produce poorer text.
- It skips a page silently if OCR returns nothing.
- It does not attempt to preserve reading order, columns, or layout.
- Tesseract does not need an internet connection or an API key - it runs locally - which is a plus, but it also means quality is limited to what the local engine can do.

Chapter 10. Table Extraction Explained

10.1 Why tables need special handling

A table's meaning lives in its layout. In “Glucose 95”, the number 95 means nothing without knowing that the column header is “Result” and the row label is “Glucose”. But an embedding model reads text as a sequence of words; a raw grid of cells would lose the relationship between a heading and its values. So the extractor rewrites each row as explicit `header: value` pairs.

10.2 How the current extractor works

`extract_tables_from_pdf` uses **pdfplumber**, which analyses the lines and positions on a page. It iterates pages, calls `page.extract_tables()`, and for each detected table:

10.3 Headers and rows

```
        headers = cleaned_rows[0]      # ["Test", "Result", "Unit"]
        data_rows = cleaned_rows[1:]   # [["Glucose", "95", "mg/dL"],
                                       #  ["Cholesterol", "180", "mg/dL"]]
```

10.4 How a table becomes searchable text

```
        for row in data_rows:
            values = []
            for index, header in enumerate(headers):
                value = row[index] if index < len(row) else ""
                values.append(f"{header}: {value}")
            searchable_rows.append(" | ".join(values))

        content = "\n".join(searchable_rows)
```

10.5 The same table at every stage

Here is one small table followed through the pipeline.

```
STAGE 1 - as it appears on the page
    +---------------+---------+--------+
    | Test          | Result  | Unit   |
    +---------------+---------+--------+
    | Glucose       | 95      | mg/dL  |
    | Cholesterol   | 180     | mg/dL  |
    +---------------+---------+--------+

STAGE 2 - after pdfplumber (list of lists)
    [["Test","Result","Unit"],
     ["Glucose","95","mg/dL"],
     ["Cholesterol","180","mg/dL"]]

STAGE 3 - after cleaning + split into headers and rows
    headers = ["Test","Result","Unit"]
    rows    = [["Glucose","95","mg/dL"],
               ["Cholesterol","180","mg/dL"]]

STAGE 4 - searchable text (this is what gets embedded)
    "Test: Glucose | Result: 95 | Unit: mg/dL
     Test: Cholesterol | Result: 180 | Unit: mg/dL"

STAGE 5 - content unit
    { content_type: "table", page_number: n, table_number: k,
      content: , metadata: { headers, rows, source } }

STAGE 6 - Pinecone record
    metadata: { document_id, content_type="table", page_number,
                content=, table_number, source, content_ref="" }
    (note: the headers/rows lists are NOT stored in Pinecone; only the
     searchable text carries the headers as part of each row)
```

Figure 10.1 - a table through every stage

10.6 Why both forms matter

Limitation: Only the searchable text and the table_number reach Pinecone. If you needed the full grid at answer time, you would have to re-read the source document; the vector store does not return the rows list. Also, header detection is naive: the first row is always assumed to be a header.

Chapter 11. Image Extraction and Image Storage

11.1 How images are extracted

Images are pulled out with **PyMuPDF** (imported as `fitz`). For each page the code calls `page.get_images(full=True)`, which returns a list of images embedded on that page. Each entry begins with an internal reference number called an **xref**. PyMuPDF's `extract_image(xref)` then returns the raw image bytes and its format (for example png or jpeg).

11.2 What an image reference is

An **image reference** (stored as `content_ref`) is a short string identifying where the image file was saved, expressed relative to the storage folder and using forward slashes. Example: `images/0b3ccb21-.../page_2_img_1.png`. It is not the image itself; it is a signpost to the image.

11.3 Where image files are stored, and how the identifiers relate

```
storage/
  images/
    0b3ccb21-91eb-441d-af84-ee9403407643/        <-- one folder per document_id
      page_1_img_1.png
      page_2_img_1.jpeg
      page_2_img_2.png
```

Figure 11.1 - image storage layout

Identifier	Example	Meaning
document_id	0b3ccb21-...	Which document this image belongs to.
page_number	2	Which page it was embedded on.
image_index	1	Which image on that page (1-based).
image_id	0b3ccb21-..._image_3	A human-readable id; the number is a document-wide counter.
content_ref	images/0b3ccb21-.../page_2_img_1.png	The relative file path used for storage and later resolution.
image_format	png	The file extension returned by PyMuPDF.

11.4 Why an image file is not the same as its metadata

The **file** is pixels. The **metadata** is text about the file (its id, page, format, and reference). Pinecone stores only the metadata text, never the pixels. That is why the system can *find* an image by searching, yet still needs a separate step (`resolve_image_path`) to obtain the actual pixels for a vision model.

11.5 How an image becomes a content unit

```
        image_unit = {
            "document_id": document_id,
            "content_type": "image",
            "page_number": page_number,
            "image_id": f"{document_id}_image_{image_counter}",
            "content_ref": "images//page__img_.",
            "image_format": image_extension,
            "source": "pdf_image",
        }
```

`build_content_units` then wraps this into a content unit whose `content` is empty and whose `content_ref` points at the file. The `metadata` keeps the image_id and image_format.

11.6 How a reference becomes a real path

```
        content_ref = "images//page_2_img_1.png"
        BASE_DIR    =           # from engine.py location
        resolved    = (BASE_DIR / "storage" / content_ref).resolve()
        # -> C:\...\backend\storage\images\\page_2_img_1.png
```

resolve_image_path() in engine.py

If the resolved file does not exist, `resolve_image_path` raises `FileNotFoundError`. That is exactly the error the multimodal test script checks for, because an image that cannot be found cannot be shown to a vision model.

11.7 Limitations of the current extraction

- Only images embedded as separate objects are extracted. Vector graphics, charts drawn with PDF drawing commands, and most “figures” composed of shapes and text are **not** captured.
- Repeated use of the same image across pages may be written as separate files each time, because the loop counts per page.
- There is no de-duplication, no resizing, and no thumbnail generation.
- The image is never analysed visually during indexing - only its metadata sentence is embedded (see Chapter 12).

Chapter 12. Embeddings Explained from Zero

12.1 What a vector is

A **vector** is simply an ordered list of numbers, like `[0.12, -0.98, 0.44, ...]`. Each position is called a **dimension**. A vector with 1024 numbers has 1024 dimensions.

12.2 What an embedding is

An **embedding** is a vector that represents the *meaning* of a piece of text (or an image). An embedding model is trained so that texts with similar meaning produce vectors that point in similar directions. You never read the numbers yourself; you use them to measure similarity.

12.3 Why text is converted into numbers

Computers compare numbers easily but not meanings. Turning text into numbers lets the computer compute a distance between two pieces of text. Small distance means similar meaning.

12.4 Semantic similarity vs. exact keyword matching

Keyword matching looks for the literal letters. If you search for “car”, a page about “automobile” is a miss. **Semantic** (meaning-based) matching compares embeddings, so “car” and “automobile” land close together. This is why the project can answer a question phrased differently from the document's wording.

12.5 Dimensions, and why query and documents must match

The embedding model returns a fixed number of dimensions, set here to **1024**. A vector database index is created for a specific dimension count. If you embedded documents with one model (say 768 dimensions) and then searched with another (say 1024), the two spaces would be incompatible and the comparison meaningless. Rule: **the same model and the same dimension must be used for both indexing and querying**.

12.6 The actual Gemini integration

```
        client = genai.Client(api_key=GEMINI_API_KEY)
        EMBEDDING_MODEL = "gemini-embedding-001"

        response = client.models.embed_content(
            model=EMBEDDING_MODEL,
            contents=text,
            config=types.EmbedContentConfig(output_dimensionality=1024),
        )
        return response.embeddings[0].values
```

The model is **gemini-embedding-001**. The output dimensionality is explicitly set to 1024. Both index-time and query-time embeddings go through this same function, so their spaces match.

12.7 How text, tables and images are embedded (the honest version)

Content type	Text that is embedded	Kind of embedding
text	the extracted page text itself	true semantic text embedding

Content type	Text that is embedded	Kind of embedding
table	the header: value searchable rows	true semantic text embedding
image	`"Image <image_id> extracted from page <n> of the document."`	**NOT visual** - a metadata sentence

> **This is the single most important honesty point in the whole guide:** The code contains **no** true visual embedding. `get_embedding_text()` returns a fixed English sentence for every image, so all images on the same page produce *near-identical* embeddings. Image "search" therefore mostly reflects the page number, not the picture's content. The picture itself is only seen later, when its file path is passed to the vision model. The stale `test_visual_embedding.py` (which imports a `generate_image_embedding` function that does not exist) is evidence that a visual embedding was once attempted but is not present in the current code.

12.8 How a vector is associated with a content unit

```
        embedded_unit = { **content_unit, "embedding": embedding,
                          "embedding_text": embedding_text }
        # later, in pinecone_service.upsert_content_units:
        vector = { "id": unit["id"], "values": unit["embedding"], "metadata": {...} }
```

The association is by `id`. The content unit's `id` (a UUID) becomes the Pinecone vector id, so a search hit can be matched back to the exact content unit's metadata.

Chapter 13. Pinecone Explained

13.1 Why a vector database is used

A normal database finds rows by exact conditions (id = 5, name = 'Ana'). A **vector database** finds items by *similarity*: given a query vector, it returns the stored vectors that are closest to it. That is exactly what RAG retrieval needs. Pinecone is a hosted vector database, so you do not run it yourself; you send vectors and queries over the internet with an API key.

13.2 Index, namespace, ids, values, metadata

Term	Meaning in this project
Index	A named container where vectors live. Name comes from `PINECONE_INDEX_NAME`, default `context-engineering-harness`.
Namespace	A partition inside an index. Here it is hard-coded to `multimodal-v2`, deliberately separating these vectors from older ones.
Vector id	Each vector's unique name. Here it is the content unit's UUID.
Vector values	The 1024 numbers produced by the embedding model.
Metadata	A small dictionary of text/numbers attached to a vector: document_id, content_type, page_number, content, content_ref, source (+ table_number/image_id).

13.3 How upsert works

“Upsert” means “insert or update”. `upsert_content_units` builds a list of vector dictionaries and sends them in one call. If an id already exists, its values and metadata are replaced; otherwise a new record is created. Sending vectors in a batch is much faster than one at a time.

13.4 How similarity search works

The query text is embedded with the same model, giving a query vector. Pinecone compares it with stored vectors and ranks them by distance. **Top-k** means “return the k closest ones” - here k is the `top_k` value (default 5, maximum 20). `include_metadata=True` asks Pinecone to return each match's metadata along with its id and score.

13.5 Document filters

```
        search_filter = None
        if document_id:
            search_filter = {"document_id": {"$eq": document_id}}
```

Optional metadata filter

When a document_id is supplied, search is restricted to records whose metadata `document_id` equals it. `$eq` is Pinecone's “equals” operator. This is how you ask about one document and ignore the rest.

13.6 What results contain, and how the real content is retrieved

```
        result = {
            "id": match.id,
            "score": match.score,
            "document_id": ..., "content_type": ..., "page_number": ...,
            "content": metadata.get("content", ""),
            "content_ref": metadata.get("content_ref", ""),
            "source": ...,
        }
        # plus "table_number" or "image_id" when present
```

The actual content is stored *inside the metadata's* `content` field, so a search result already carries the text - there is no second lookup. For an image, the result carries a `content_ref` rather than pixels, so the real file must be loaded separately via `resolve_image_path`.

13.7 Index configuration requirements

- **Dimension:** must be **1024**, because the embedding model is asked for 1024-dimensional vectors.
- **Metric:** the code does not state the similarity metric; it only relies on a `score` field. The metric (commonly cosine) must be chosen when the index is created and cannot be verified from the repository.
- **Namespace:** `multimodal-v2` is used for both upsert and query, so they stay consistent.
- The index is expected to already exist; the code connects to it rather than creating it.

> **Could not verify:** The similarity metric and whether the index was created with 1024 dimensions cannot be confirmed from the code alone - there is no index-creation script in the repository. The code's behaviour implies 1024 dimensions, because that is what it sends and queries with.

13.8 Migration / re-indexing note

The namespace `multimodal-v2` and its comment (“keep new multimodal vectors isolated from old vectors”) indicate that the embedding or content scheme changed at some point. If the embedding model or its dimensionality ever changes again, the correct action is to re-embed and re-upsert all documents into a fresh namespace, because old vectors would no longer be comparable with new queries.

Chapter 14. Retrieval-Augmented Generation (RAG)

14.1 RAG from absolute beginner level

Think of a student taking an open-book exam.

Letter	Word	In this project
R	Retrieval - find the relevant pages	`retrieve_context()` embeds the question and searches Pinecone
A	Augmentation - add those pages to the question	`format_context()` turns results into prompt text
G	Generation - produce the answer	`generate_response()` or `generate_multimodal_response()`

14.2 The actual implementation

The whole flow lives in two files: `rag_service.py` (retrieval) and `context_engine/engine.py` (assembly and formatting), with `main.py` tying them together and choosing the model.

14.3 One question, followed all the way

```
QUESTION: "What was the glucose result?"

(1) main.ask_document receives RAGRequest(query=..., top_k=5, document_id=None)
    |
(2) engine.assemble_context(query, 5, None)
    |
(3) rag_service.retrieve_context(query, 5, None)
    |
(4) embedding_service.generate_text_embedding("What was the glucose result?")
      -> generate_embedding(...) -> Gemini -> 1024-number vector Q
    |
(5) pinecone_service.search_similar_content(Q, 5, None)
      -> index.query(vector=Q, top_k=5, include_metadata=True,
                     namespace="multimodal-v2", filter=None)
      -> matches, e.g.:
         [ {score:0.81, content_type:"table", page:1,
            content:"Test: Glucose | Result: 95 | Unit: mg/dL"},
           {score:0.63, content_type:"text",  page:1,
            content:"Lab Results ..."},
           ... ]
    |
(6) assemble_context returns { conversation:None, memory:[],
                               rag:[...], knowledge_graph:[] }
    |
(7) engine.format_context(context) produces:
       "[TABLE - Page 1]
        Test: Glucose | Result: 95 | Unit: mg/dL

        [TEXT - Page 1]
        Lab Results ..."
    |
(8) no image among the results -> llm.generate_response(prompt)
    |
(9) Ollama llama3.2 answers using the injected context
    |
(10) JSON { query, answer, context, images_sent_to_llm: 0 }
```

Figure 14.1 - one RAG question end to end

14.4 How text, tables and images are treated

Type	In retrieval	In the prompt
Text	Found by semantic similarity over extracted text.	Inserted verbatim under [TEXT - Page n].
Table	Found via the searchable header:value text.	Inserted under [TABLE - Page n].
Image	Found via the metadata sentence (not the picture).	Represented as [IMAGE - Page n] with a file reference; the pixels are attached separately for the vision model.

14.5 Finding an image vs. sending an image to a model

These two things must not be confused:

- **Finding (retrieval):** happens inside Pinecone, working on numbers. For images these numbers came from a text sentence, so "finding" an image is really finding its metadata.
- **Sending (vision):** happens in `generate_multimodal_response`, where the real files at their absolute paths are attached to the Ollama request. Only then does a model actually look at the picture.

14.6 What remains incomplete about image-to-LLM integration

The image-to-LLM path exists in code and is exercised by `test_multimodal_rag.py`, but a few things are worth stating plainly:

- The React interface never triggers `/documents/ask`, so in normal use no image is ever sent to the vision model.
- Which images are sent depends on the *metadata-sentence* similarity, which is weak, so the "right" image may not be the one retrieved.
- Images are only found if they were embedded as separate objects; figure-like drawings are missed entirely (Chapter 11).
- There is no cap on how many images are sent for `/documents/ask` (the test script caps at three, but the endpoint does not), which could bloat the request.
- The vision model tag `qwen3.5:4b` must exist locally in Ollama, or the call fails with an **LLMError**.

Chapter 15. Context Engine Explained

15.1 Why a context engine is useful

A language model only sees one thing: the text you give it. Deciding *what* goes into that text is a real design problem. A **context engine** is the single component that collects everything a model needs - retrieved documents, past conversation, stored facts - and assembles one coherent context. Keeping this in one place means the answer quality does not depend on each endpoint reinventing the prompt.

15.2 Retrieval vs. context assembly

	Retrieval	Context assembly
Question it answers	“Which stored items are relevant?”	“What exactly do we send to the model?”
Code	`retrieve_context()` in rag_service.py	`assemble_context()` and `format_context()` in engine.py
Output	A list of result dictionaries	A context dictionary, then a prompt string

15.3 What assemble_context() actually does

```
        def assemble_context(query, top_k=5, document_id=None) -> dict:
            rag_results = retrieve_context(query=query, top_k=top_k,
                                           document_id=document_id)
            context = {
                "conversation": None,
                "memory": [],
                "rag": rag_results,
                "knowledge_graph": [],
            }
            return context
```

In the current implementation it does two things: it calls retrieval, and it places the results into a fixed four-key dictionary. That is the entire function.

15.4 What format_context() actually does

It walks `context['rag']` and builds one string. Each result becomes a labelled block: `[TEXT - Page n]`, `[TABLE - Page n]`, or `[IMAGE - Page n]`. Text and table blocks include the stored content; the image block includes only the reference. Blocks are separated by blank lines. This string is then inserted into the prompt in `ask_document`.

15.5 How image references are represented

Inside the context dictionary, an image appears as a result with `content_type='image'` and a `content_ref` string. In the formatted text it becomes a line “Image reference: images/.../page_n_img_k.ext”. The actual pixels never enter the context dictionary or the formatted text; they enter only when `ask_document` resolves the reference to an absolute path and passes that path to the vision model.

15.6 How the Context Engine connects to RAG and the LLM

```
context_engine.engine  (assemble + format)
      |  calls  ->  rag_service.retrieve_context      (RAG)
      |                    |  calls -> embedding_service + pinecone_service
      |  feeds  ->  main.ask_document -> llm.generate_response
      |                                -> llm.generate_multimodal_response
```

Figure 15.1 - the Context Engine sits between RAG and the LLM

15.7 What the current context dictionary contains

Key	Current value	Status
conversation	`None`	Placeholder - not populated
memory	`[]` (always empty)	Placeholder - not implemented
rag	The list of retrieval results	Implemented
knowledge_graph	`[]` (always empty)	Placeholder - not implemented

15.8 Implemented vs. planned context components

- **Implemented:** the `rag` component, plus image path resolution and prompt formatting.
- **Planned (not implemented):** short-term memory (the current turn-to-turn conversation), long-term memory, conversation summaries, and a knowledge graph. The empty keys are reserved space for these ideas, but no code fills them.

> **Be precise about memory:** There is no conversational memory anywhere in this project. The React app keeps a list of displayed messages, but it sends only the newest message to `/chat`, and the backend stores nothing. `/documents/ask` is single-turn as well.

15.9 Why one authoritative context path is a good principle

If each endpoint built its own prompt, they would drift apart, and improvements (like adding memory) would have to be made in many places. Funnelling every model call through one context builder means there is a single place to reason about what the model sees. The current code follows this principle only partly: `/chat` bypasses the Context Engine entirely and calls the model directly, so the principle is an aspiration that is only realised on the document-question path.

Chapter 16. Frontend Explained

16.1 Components and imports

The frontend has one real component, **App**, in `src/App.jsx`. It imports `useState` from React and `./App.css`. The entry file `src/main.jsx` imports `React.StrictMode`, `createRoot`, `./index.css`, and **App**, then mounts it into the `#root` element from `index.html`.

```
// main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(


  ,
)
```

16.2 React state

All state lives in **App**. There are seven variables (listed in Chapter 5). They cover chat (message, messages, loading) and upload (selectedFile, documentId, uploading, uploadError). There is no router, no context provider, and no external state library.

16.3 Controlled inputs

The chat textarea is a **controlled input**: its `value` is `{message}` and its `onChange` calls `setMessage`. The value shown on screen is therefore always exactly the state - React and the DOM never disagree. The file input is uncontrolled (the browser manages it); the code reads the file from the change event.

16.4 Chat messages, loading indicators and errors

Messages are rendered by mapping the array. The CSS class is `` `message ${item.role}` ``, so `.message.user` and `.message.assistant` can be styled differently. While `loading` is true, a temporary assistant bubble shows “Thinking...”. Errors do not use a separate area: an error is appended as an assistant message containing the error text.

16.5 PDF selection and upload validation

`handleFileChange` clears previous state, then accepts a file only if `file.type === 'application/pdf'` or its name ends with `.pdf` (case-insensitive). Rejected files clear the selection and set `uploadError` to “Please select a PDF file.”

16.6 FormData and the upload request

```
const formData = new FormData();
formData.append("file", selectedFile);       // field name must be "file"
const response = await fetch("http://127.0.0.1:8000/documents/upload", {
  method: "POST",
  body: formData,                            // NOTE: no Content-Type header
});
```

FormData is the browser's way of building a form submission, including file bytes. Crucially, the code does **not** set `Content-Type` here: the browser sets it automatically, including the multipart boundary.

Setting it manually would break the upload. The field name `"file"` must match FastAPI's `File(...)` parameter name.

16.7 API responses and how the UI renders them

- Chat success → `data.response` becomes an assistant bubble.
- Upload success → `data.document_id` is stored and shown, with the line “Document uploaded and processed successfully.”
- Both paths check `response.ok`; on failure they throw using the server's `detail` field.
- The `document_id` shown on screen is **not** used by the chat feature - it is informational only.

16.8 CSS layout and responsiveness

`App.css` centres a card layout (`.app`, max width 1000px) with two white cards: `.upload-section` and `.chat-section`. Buttons are blue with a grey disabled state. Chat bubbles use `max-width: 80%` with user bubbles pushed right (`margin-left:auto`) and assistant bubbles left. A `@media (max-width:700px)` rule reduces padding, shrinks the title, stacks the chat controls vertically, and widens bubbles to 95%.

`index.css` is the Vite template base sheet. It defines colour variables, a dark-mode block via `prefers-color-scheme`, and constrains `#root` to 1126px. Both stylesheets are loaded, so a few base rules from the template also apply.

16.9 Why the frontend never touches Pinecone or Gemini directly

Two reasons. First, **secrets**: API keys would have to live in the browser, where anyone could read them; keeping keys on the server protects them. Second, **control**: the backend decides the pipeline (which pages to OCR, how to shape tables, which model to call). The browser only sends a message or a file and displays the result. Always calling `http://127.0.0.1:8000` keeps the browser simple.

Chapter 17. API Reference

This chapter documents every endpoint that actually exists in `app/main.py`. Endpoints that are commonly assumed but absent are listed at the end.

17.1 GET /

Field	Detail
Method & path	GET /
Purpose	Health check - confirms the server is running.
Request	None.
Response	`{ "message": "Context Engineering Harness API is running." }`
Functions called	none
Status codes	200

17.2 POST /chat

Field	Detail
Purpose	Send a plain text message to llama3.2 and get a reply.
Request body	**ChatRequest**: `{ "message": str }` (min length 1).
Response body	**ChatResponse**: `{ "response": str }`.
Functions called	`generate_response()` → `ollama.chat(llama3.2)`
Status codes	200 OK; 422 unprocessable (validation); 503 if the model fails (**LLMError**).

```
# Request
{ "message": "Hello, who are you?" }

# Success response
{ "response": "I am a language model running locally..." }

# LLM failure response (HTTP 503)
{ "detail": "The language model could not generate a response: ..." }
```

17.3 POST /documents/upload

Field	Detail
Purpose	Store a PDF and run the full extraction/indexing pipeline.
Request	multipart/form-data with one file field named `file`.
Response body	**DocumentUploadResponse**: document_id, filename, status, text_units, ocr_pages, tables, images.

Field	Detail
Functions called	`save_document()`, then `process_document()` (which calls every extractor, the chunker, the embedding service and Pinecone).
Status codes	200 OK; 400 for a non-PDF (**ValueError**); 500 if a dependency (e.g. Gemini/Pinecone) fails during processing.

```
# curl example
curl -X POST http://127.0.0.1:8000/documents/upload \
     -F "file=@report.pdf"

# Response
{ "document_id": "0b3ccb21-91eb-441d-af84-ee9403407643",
  "filename": "report.pdf", "status": "processed",
  "text_units": 12, "ocr_pages": 2, "tables": 3, "images": 5 }
```

17.4 POST /documents/extract-preview

Field	Detail
Purpose	Development inspection: run the pipeline and return the full extraction detail.
Request	multipart/form-data with one file field named `file`.
Response body	`{ document_id, filename, content_units, ocr_pages, tables, images }` (plain dict, not a schema).
Functions called	same as upload: `save_document()` + `process_document()`.
Status codes	200 OK; 400 for invalid files.
Side effect	Like upload, it embeds and upserts to Pinecone - it is not read-only.

17.5 POST /documents/search

Field	Detail
Purpose	Semantic search over indexed content, without any language model.
Request body	**DocumentSearchRequest**: `{ query: str, document_id?: str|null, top_k?: int (1..20, default 5) }`
Response body	`{ "query": str, "results": [ {id, score, document_id, content_type, page_number, content, content_ref, source, [table_number|image_id]} ] }`
Functions called	`generate_embedding()`, `search_similar_content()`
Status codes	200 OK; 400 if the query is empty (**ValueError**); 422 on validation.

```
# Request
{ "query": "glucose result", "top_k": 3 }

# Response
{ "query": "glucose result",
  "results": [
    { "id": "a1...", "score": 0.81, "document_id": "0b3c...",
      "content_type": "table", "page_number": 1,
      "content": "Test: Glucose | Result: 95 | Unit: mg/dL",
      "content_ref": "", "source": "pdf_table", "table_number": 1 }
  ] }
```

17.6 POST /documents/ask

Field	Detail
Purpose	Full RAG: retrieve, assemble context, optionally attach images, and generate an answer.
Request body	**RAGRequest**: `{ query: str, document_id?: str|null, top_k?: int (1..20, default 5) }`
Response body	`{ "query": str, "answer": str, "context": dict, "images_sent_to_llm": int }`
Functions called	`assemble_context()`, `format_context()`, `resolve_image_path()`, then `generate_multimodal_response()` (if images) or `generate_response()`.
Status codes	200 OK; 400 (**ValueError**); 503 (**LLMError**); 500 if an image file is missing (`FileNotFoundError` is not caught).

```
# Request
{ "query": "What was the glucose result?", "top_k": 5 }

# Response (no image retrieved -> llama3.2)
{ "query": "What was the glucose result?",
  "answer": "The glucose result was 95 mg/dL.",
  "context": { "conversation": null, "memory": [],
               "rag": [ ... ], "knowledge_graph": [] },
  "images_sent_to_llm": 0 }
```

17.7 Endpoints that do NOT exist

- There is no `/documents/list`, no delete endpoint, and no endpoint that returns a stored document by id.
- There is no endpoint to fetch a saved image; images are only read from disk internally.
- There is no streaming endpoint and no WebSocket.
- There is no authentication or user endpoint.

Chapter 18. Configuration, Dependencies, and Environment

18.1 The Python virtual environment

A **virtual environment** is a private folder of Python packages for one project. It prevents one project's libraries from clashing with another's. This project uses `backend/.venv`. Activate it before running anything:

```
cd backend
source .venv/Scripts/activate      # Git Bash on Windows
```

The environment currently contains FastAPI 0.142.2, Uvicorn 0.54.0, Ollama 0.6.3, PyMuPDF 1.28.2, pdfplumber 0.11.10, pytesseract 0.3.13, Pillow 12.3.0, google-genai 2.29.0, pinecone 10.0.0, python-dotenv 1.2.4, python-multipart 0.0.32, and pydantic 2.13.5.

18.2 requirements.txt

The file lists pinned versions. Two observations from reading it:

- It appears to be saved in a UTF-16 encoding (each character is followed by a null byte when read as plain text), which some tools handle and others do not.
- It contains the dependency list **twice**: a fully pinned block (with ==versions) and then a looser second block of bare names (fastapi, uvicorn, python-dotenv, ollama, python-multipart, pymupdf, pytesseract, pillow, pdfplumber, google-genai, pinecone).
- The important packages for the AI features are **google-genai** (Gemini embeddings) and **pinecone** (vector database), which are present in the pinned block.

> **Not in requirements.txt:** Tesseract itself is a separate system program, not a Python package. `pytesseract` only calls it. Tesseract must be installed separately for OCR to work.

18.3 Environment variables

Environment variables are settings read at runtime. This project reads them from a `.env` file at the **project root** (the same folder as `backend` and `frontend`), loaded by `python-dotenv`. Secret values are deliberately not printed in this guide.

Variable	Read by	Purpose	Required?
GEMINI_API_KEY	embedding_service.py	Authenticates calls to Google Gemini for embeddings.	Yes - the module raises RuntimeError at import if missing.
PINECONE_API_KEY	pinecone_service.py	Authenticates calls to Pinecone.	Yes - the module raises RuntimeError at import if missing.
PINECONE_INDEX_NAME	pinecone_service.py	The index to use. Defaults to `context-engineering-harness` if unset.	No (has a default)

Variable	Read by	Purpose	Required?
TESSERACT_PATH	ocr.py	Location of the Tesseract program. Defaults to the Windows install path.	No (has a default)

> **Note:** `TESSERACT_PATH` is read with `os.getenv` but is **not** present in the current `.env`, so the built-in Windows default is used.

18.4 Where .env must be placed, and why

Both `embedding_service.py` and `pinecone_service.py` call `load_dotenv()` with no path, so python-dotenv searches upward from the current working directory for a `.env`. In practice this means the backend must be started from a location where the root `.env` is discoverable. If the keys are not found, both modules fail at import and the whole server refuses to start.

18.5 Gemini, Pinecone and Ollama configuration

Service	What configuration is needed	Failure mode if missing
Gemini	A valid `GEMINI_API_KEY` and internet access.	RuntimeError at import; or a request error during embedding.
Pinecone	A valid `PINECONE_API_KEY`, an existing index (dimension 1024), and network access.	RuntimeError at import; or a query/upsert error.
Ollama	The Ollama service running locally with the models **llama3.2** and **qwen3.5:4b** pulled.	LLMError → HTTP 503 on chat/ask.
Tesseract	The Tesseract program installed at the configured path.	OCR raises an error when a scanned page is processed.

18.6 Frontend dependencies

`package.json` lists react and react-dom (^19.2.8) as runtime dependencies, and vite (^8.3.0), the react plugin, oxlint, and the React type packages as development dependencies. There is no HTTP library (the built-in fetch is used) and no UI library (plain CSS is used).

18.7 Local storage paths

Path	Created by	Contents
backend/storage/documents/	`save_document()` (mkdir)	One PDF per upload, named <document_id>.pdf
backend/storage/images/	`extract_images_from_pdf()` (mkdir)	Per-document sub-folders of extracted images

These folders currently contain real data (many stored PDFs and per-document image folders), which is evidence the pipeline has been run repeatedly during development. This guide does not modify them.

Chapter 19. Testing and Debugging

19.1 What tests exist

All tests are loose scripts in `backend/`. They are **not** a test framework like pytest; most run code at import and print results. They must be run from the `backend` folder with the virtual environment active, because they import `app.rag...` modules.

Script	What it verifies	What it does NOT verify
test_embedding.py	That Gemini returns an embedding and its length; prints the first five values.	Dimension correctness against Pinecone, or quality.
test_indexing.py	That one hand-made text unit can be embedded and upserted; prints the vector count.	Retrieval of that vector; metadata completeness.
test_pinecone.py	Index connectivity and statistics via describe_index_stats().	Content correctness.
test_retrieval.py	Similarity search returns results with score/type/page/content.	That the top result is actually the right one.
test_multimodal_rag.py	Retrieval, that image refs resolve to real files (asserts existence), and that qwen3.5:4b can describe a retrieved image.	End-to-end UI behaviour; it is interactive (asks for a document id).
test_visual_embedding.py	Intended to test a true visual image embedding.	BROKEN: imports generate_image_embedding, which does not exist, so it fails with ImportError.

> **Do not claim these pass:** This guide was produced by reading the code, not by running the scripts (running them would call paid/remote services and require interactive input). Their current pass/fail status could not be verified here. In particular, `test_visual_embedding.py` will fail immediately, and the integration scripts require live Gemini, Pinecone, Ollama and Tesseract.

19.2 How to run them

```
cd backend
source .venv/Scripts/activate
python test_embedding.py          # prints dimensions and first values
python test_pinecone.py           # prints index statistics
python test_indexing.py           # upserts one vector
python test_retrieval.py          # prints search results
python test_multimodal_rag.py     # interactive: asks for a document id
```

Running the manual tests

19.3 Verifying each capability by hand

Capability	How to verify
API endpoints	Start the server and open `http://127.0.0.1:8000/docs` (FastAPI's auto page), or use the curl examples in Chapter 17.
PDF extraction	Call `/documents/extract-preview` and inspect `content_units` and the `source` values.
OCR	Upload a scanned PDF and check that some units have `source: "ocr"` in the preview; also check `ocr_pages` in the upload response.
Table extraction	In the preview, look for units with `content_type: "table"` and confirm `metadata.headers`/`rows` and the header:value `content`.
Image extraction	In the preview, look for `content_type: "image"` units, then confirm the files exist under `backend/storage/images/<id>/`.
Embeddings	Run `test_embedding.py`; it should report 1024 dimensions.
Pinecone indexing	Run `test_indexing.py`, then `test_pinecone.py` to see the vector count change.
Pinecone retrieval	Run `test_retrieval.py` and inspect the printed scores.
Image actually reaching the vision model	Run `test_multimodal_rag.py`; its assertions confirm the image files exist and the model returns non-empty text. The endpoint also reports `images_sent_to_llm`.

19.4 Debugging guide for common failures

Symptom	Likely cause	What to check
Server will not start; RuntimeError about a key	GEMINI_API_KEY or PINECONE_API_KEY missing/unreadable.	Confirm .env location and that the keys exist (without printing them).
Chat returns 503	Ollama not running, or llama3.2 not pulled.	Start Ollama; run `ollama pull llama3.2`.
Multimodal call fails	qwen3.5:4b not available locally.	Run `ollama pull qwen3.5:4b` (matching the tag in code).
Upload returns 400	The file is not a .pdf.	Check the extension; the server only accepts .pdf.
Upload returns 500	Embedding or Pinecone error mid-pipeline.	Check API keys, quota, network, and the index's dimension (must be 1024).
OCR error	Tesseract not installed or wrong path.	Install Tesseract or set TESSERACT_PATH.
Image resolution fails (FileNotFoundError)	Stored image file missing, or a bad content_ref.	Check backend/storage/images/<id>/ and the ref string.

Symptom	Likely cause	What to check
Search returns odd results	Images match by a metadata sentence, not by picture.	Expect weak image matches; verify text/table matches first.

19.5 What is missing from testing

- No unit tests for `is_meaningful_text`, `table_to_searchable_text`, `clean_cell`, `build_content_units`, or `format_context` - all of which are pure functions and easy to test.
- No API-level tests (FastAPI's TestClient is not used).
- No test asserts that an image actually influenced a vision-model answer (only that an image was sent).
- No frontend tests.

Chapter 20. End-to-End Execution Traces

Each trace shows the starting event, the function called, the input, the processing, the output, the next function, and the final result.

20.1 Sending a normal chat message

```
START     User types "Explain retrieval" and presses Enter
CALLED    App.sendMessage()
INPUT     message = "Explain retrieval"
PROCESS   append user bubble; clear box; setLoading(true)
CALLED    fetch POST http://127.0.0.1:8000/chat
INPUT     body { "message": "Explain retrieval" }
CALLED    FastAPI validates ChatRequest
CALLED    main.chat(request)
CALLED    llm.generate_response("Explain retrieval")
PROCESS   ollama.chat(model="llama3.2")
OUTPUT    "Retrieval is the process of finding relevant information..."
RETURN    ChatResponse(response="Retrieval is ...")
CALLED    React setMessages(appends assistant bubble); setLoading(false)
RESULT    An assistant bubble shows the reply.
```

Trace 1 - normal chat

20.2 Uploading a text-based PDF

```
START     User selects report.pdf (born-digital) and clicks Upload PDF
CALLED    App.handleUpload()
INPUT     FormData { file: report.pdf }
CALLED    POST /documents/upload
CALLED    save_document(file)
PROCESS   create storage/documents/.pdf
OUTPUT    (document_id, "report.pdf", path)
CALLED    process_document(path, id)
CALLED    extract_text_from_pdf -> text_units (source "pdf_text"),
                                   pages_needing_ocr = []  (all pages fine)
CALLED    extract_tables_from_pdf -> table_units
CALLED    extract_images_from_pdf -> image_units (+ files)
CALLED    build_content_units -> content_units (sorted)
CALLED    embed_content_units -> embedded_units (1024-dim each)
CALLED    upsert_content_units -> index.upsert(namespace="multimodal-v2")
OUTPUT    DocumentUploadResponse(counts...)
CALLED    React setDocumentId(data.document_id)
RESULT    The document id is shown; vectors are now in Pinecone.
```

Trace 2 - text PDF

20.3 Uploading a scanned PDF requiring OCR

```
START     User uploads scan.pdf (images of pages)
PROCESS   text extraction finds little text on pages 1..4
CALLED    extract_text_from_pdf -> pages_needing_ocr = [1,2,3,4]
CALLED    extract_text_with_ocr(path, id, [1,2,3,4])
PROCESS   for each page: render Matrix(2,2) PNG -> PIL -> pytesseract
OUTPUT    text_units with source "ocr" (pages where OCR found text)
CALLED    build_content_units -> embed_content_units -> upsert_content_units
RESULT    Upload response shows ocr_pages = 4; the scanned text is searchable.
```

Trace 3 - scanned PDF

20.4 Uploading a PDF containing a table

```
CALLED    extract_tables_from_pdf(path, id)
INPUT     page 1 with a 3-column table
PROCESS   pdfplumber.extract_tables -> rows ; clean cells ;
          headers = row[0] ; data = rows[1:]
OUTPUT    content = "Test: Glucose | Result: 95 | Unit: mg/dL"
          table_unit { page_number, table_number, headers, rows, content }
CALLED    build_content_units -> table content unit (metadata keeps headers/rows)
CALLED    embed_content_units -> embedding of the header:value text
CALLED    upsert -> metadata { content: , table_number, ... }
RESULT    The table is findable by asking "glucose result".
```

Trace 4 - table PDF

20.5 Uploading a PDF containing an image

```
CALLED    extract_images_from_pdf(path, id, IMAGE_STORAGE_DIR)
PROCESS   page.get_images(full=True) -> xrefs
          extract_image(xref) -> bytes + ext
          write storage/images//page_2_img_1.png
          content_ref = "images//page_2_img_1.png"
OUTPUT    image_unit { content_type "image", page 2, image_id, content_ref }
CALLED    build_content_units -> image content unit (content="")
CALLED    embed_content_units -> embeds the METADATA SENTENCE, not the pixels
CALLED    upsert -> metadata { content:"", content_ref, image_id, ... }
RESULT    The image is indexed (by metadata); the file exists on disk.
```

Trace 5 - image PDF

20.6 Asking a question that triggers RAG

```
START     POST /documents/ask { "query": "What was the glucose result?" }
CALLED    main.ask_document -> assemble_context -> retrieve_context
PROCESS   embed query -> search Pinecone (top_k=5)
OUTPUT    rag results (a table match scores highest)
CALLED    format_context -> "[TABLE - Page 1]\nTest: Glucose | Result: 95 ..."
CALLED    (no image among results) generate_response(prompt)
RESULT    answer states the glucose result; images_sent_to_llm = 0.
```

Trace 6 - RAG question

20.7 Retrieving an image and sending it to a vision model

This path is implemented in the backend and exercised by `test_multimodal_rag.py`. Because the React app never calls `/documents/ask`, reaching this path in normal use requires a tool such as `/docs` or curl.

```
START     POST /documents/ask { "query": "Describe the chart" }
CALLED    assemble_context -> retrieve_context -> Pinecone
OUTPUT    one or more results with content_type "image"
PROCESS   for each image result: resolve_image_path(content_ref)
          -> backend/storage/images//page_2_img_1.png  (must exist)
CALLED    generate_multimodal_response(prompt, [abs_path,...])
PROCESS   ollama.chat(model="qwen3.5:4b",
                      messages=[{role:"user", content:prompt,
                                 images:[abs_path,...]}])
RESULT    An answer that uses the actual pixels; images_sent_to_llm > 0.
```

Trace 7 - vision path (backend only today)

Chapter 21. Current Limitations and Future Roadmap

21.1 Status table

This table separates what is real today from what is still to be done. “Evidence” names the code that proves the row.

Feature	Current implementation	Evidence in code	Limitation / what remains
Text chat	Works	main.chat + llm.generate_response	No system prompt, no history, single-model.
PDF upload & storage	Works	document_service.save_document	Sync processing; no progress reporting.
Text extraction	Works	text_extractor.extract_text_from_pdf	One unit per page; no paragraph chunking.
OCR fallback	Works	ocr.extract_text_with_ocr	Default language, no pre-processing; needs Tesseract.
Table extraction	Works	table_extractor.extract_tables_from_pdf	First row assumed to be headers; grid not in Pinecone.
Image extraction	Works	image_extractor.extract_images_from_pdf	Only embedded objects; no vector figure capture.
Content units	Works	chunker.build_content_units	Page-level granularity; intra-page order by type.
Text/table embeddings	Works	embedding_service.embed_content_units	Depends on Gemini; 1024 dims fixed.
Visual image embeddings	NOT implemented	get_embedding_text returns a fixed sentence	Images are not embedded visually.
Pinecone indexing/search	Works	pinecone_service.upsert/search	Metric not documented; index must pre-exist at 1024 dims.
Context assembly	Minimal	context_engine.engine.assemble_context	Only rag populated; memory/graph empty.
Document Q&A in the UI	Not implemented	App.jsx has no call to /documents/ask	Backend exists; interface missing.
Image → vision model	Backend only	llm.generate_multimodal_response	No UI trigger; weak image retrieval; no image cap.
Conversation persistence	Not implemented	no storage; messages only in React state	Nothing survives a refresh.

Feature	Current implementation	Evidence in code	Limitation / what remains
Long-term memory	Not implemented	context["memory"] = []	Reserved slot only.
Conversation summaries	Not implemented	no summarisation code	Not started.
Knowledge graph (e.g. Neo4j)	Not implemented	context["knowledge_graph"] = []	Reserved slot only; no Neo4j dependency.
Model abstraction	Not implemented	model names hard-coded in llm.py	Switching models means editing code.
Sensitive-information masking	Not implemented	no masking code	Not started.
Evaluation tooling	Not implemented	no evaluation code	No way to measure answer quality.

21.2 Current functionality (verified by reading the code)

21.3 Possible future improvements

- Add the missing document-question interface and connect the shown `document_id` to it.
- Introduce real paragraph/sentence chunking with overlap for finer retrieval.
- Add true visual embeddings so images can be found by their content.
- Add a genuine multi-part context engine (conversation, memory, graph) and route `/chat` through it too.
- Move heavy processing to a background task so uploads can respond immediately.
- Introduce a small model/config abstraction so model names are not scattered.
- Add unit tests for the pure functions and API tests with FastAPI's TestClient.
- Add masking for sensitive information before embedding or logging.
- Add an evaluation harness to measure retrieval and answer quality.

> **Reminder:** These are suggestions derived from the code's shape, not claims about the author's intent. Nothing in this list is implemented today.

Chapter 22. Glossary and Learning Revision

22.1 Beginner-friendly glossary

Term	Plain meaning
API	A way for programs to talk to each other over the network, usually by sending data to a URL.
Endpoint	One specific URL + method (for example POST /chat) that an API accepts.
FastAPI	A Python library that turns functions into HTTP endpoints and validates data.
Uvicorn	The web server program that runs a FastAPI app and listens for requests.
CORS	Browser rules that decide whether a page may call an API on a different origin; controlled by allow_origins.
Pydantic schema	A class describing the required shape of data, used to validate input and output.
Embedding	A list of numbers representing the meaning of text (or an image).
Vector	An ordered list of numbers with a fixed length (its dimensions).
Dimension	One position in a vector; 1024 of them here.
Semantic similarity	Closeness of meaning, measured by comparing embeddings.
Vector database	A store that finds items by vector similarity; here it is Pinecone.
Index	A named container of vectors in Pinecone.
Namespace	A partition inside an index; here 'multimodal-v2'.
Metadata	Extra text/numbers attached to a vector (document_id, page_number, and so on).
Upsert	Insert or update a record (used for storing vectors).
top-k	How many closest results to return.
OCR	Optical Character Recognition: turning a picture of text into real text. Tesseract is used here.
RAG	Retrieval-Augmented Generation: find relevant content, add it to the prompt, then generate.
Context assembly	Deciding and building exactly what text is sent to the model.
Content unit	One uniform dictionary describing a piece of content (text, table, or image).
Multimodal	Handling more than one type of data (here: text, tables, images).
Vision model	A model that can look at images as well as text; here qwen3.5:4b.
LLM	Large Language Model: the text-generating model; here llama3.2 for text.
Ollama	A program that runs language models locally on your computer.
UUID	A random unique identifier string; used for document and unit ids.

Term	Plain meaning
metadata sentence (image)	The English sentence used to embed an image here, e.g. 'Image X extracted from page N'.

22.2 How the entire project works (short summary)

A React page talks to a FastAPI backend. Sending a message calls **llama3.2** and shows the reply. Uploading a PDF stores it, then reads text (with OCR where needed), tables, and images; each piece becomes a content unit; each unit is turned into a 1024-number embedding by Gemini and stored in Pinecone with metadata. Later, a document question is embedded, Pinecone returns the closest units, the Context Engine formats them into a prompt, and a model generates the answer - attaching real image files to a vision model when images were found. Conversations are not remembered, images are embedded by metadata and not by vision, and the document-question step is not yet reachable from the interface.

22.3 The function names to remember

Function	File	One-line job
sendMessage	App.jsx	Send a chat message
handleUpload	App.jsx	Send a PDF to the backend
chat	main.py	The /chat endpoint
upload_document	main.py	The /documents/upload endpoint
ask_document	main.py	The /documents/ask (RAG) endpoint
generate_response	llm.py	Text-only model call (llama3.2)
generate_multimodal_response	llm.py	Vision model call (qwen3.5:4b)
save_document	document_service.py	Store the PDF and make an id
process_document	document_service.py	Run the whole pipeline
extract_text_from_pdf	text_extractor.py	Read selectable text / flag OCR pages
extract_text_with_ocr	ocr.py	OCR the flagged pages
extract_tables_from_pdf	table_extractor.py	Read tables and make searchable text
extract_images_from_pdf	image_extractor.py	Save embedded images + refs
build_content_units	chunker.py	Make one uniform list of units
embed_content_units	embedding_service.py	Embed every unit with Gemini
upsert_content_units	pinecone_service.py	Store vectors in Pinecone
search_similar_content	pinecone_service.py	Similarity search in Pinecone
retrieve_context	rag_service.py	Embed the query and search

Function	File	One-line job
assemble_context	engine.py	Build the context dictionary
format_context	engine.py	Turn results into prompt text
resolve_image_path	engine.py	Turn a ref into an absolute path

22.4 Which file handles which responsibility

Responsibility	File
HTTP endpoints & CORS	backend/app/main.py
Request/response shapes	backend/app/schemas.py
Talking to language models	backend/app/llm.py
Context assembly & image resolution	backend/app/context_engine/engine.py
Pipeline orchestration & storage paths	backend/app/rag/document_service.py
Selectable text extraction	backend/app/rag/text_extractor.py
OCR	backend/app/rag/ocr.py
Table extraction	backend/app/rag/table_extractor.py
Image extraction	backend/app/rag/image_extractor.py
Unified content units	backend/app/rag/chunker.py
Embeddings	backend/app/rag/embedding_service.py
Vector store	backend/app/rag/pinecone_service.py
Retrieval glue	backend/app/rag/rag_service.py
User interface & fetch calls	frontend/src/App.jsx
UI layout & responsiveness	frontend/src/App.css

22.5 The complete request-to-response flow (revision)

```
CHAT
  user -> App.sendMessage -> POST /chat -> main.chat
       -> generate_response -> ollama llama3.2 -> ChatResponse -> UI bubble

UPLOAD
  user -> App.handleUpload -> POST /documents/upload -> main.upload_document
       -> save_document -> process_document
            -> text_extractor (+ ocr) / table_extractor / image_extractor
            -> chunker.build_content_units
            -> embedding_service.embed_content_units   (Gemini)
            -> pinecone_service.upsert_content_units   (Pinecone)
       -> DocumentUploadResponse -> UI shows document_id

DOCUMENT QUESTION (backend only today)
  client -> POST /documents/ask -> main.ask_document
         -> engine.assemble_context -> rag_service.retrieve_context
              -> generate embedding (Gemini) -> search (Pinecone)
         -> engine.format_context
         -> resolve_image_path for image hits
         -> generate_multimodal_response (qwen3.5:4b) if images
            else generate_response (llama3.2)
         -> JSON { query, answer, context, images_sent_to_llm }
```

Figure 22.1 - the whole system on one page

22.6 Final honesty notes

- Everything above was read from the actual repository. Where the code could not confirm a detail (such as the Pinecone similarity metric), the guide says so.
- **Images are not embedded visually.** This is the most common misconception to avoid.
- **The document-question and search endpoints are not used by the interface.** They work, but only if you call them directly.
- **There is no memory and no knowledge graph.** The empty context slots are placeholders, not features.
- `test_visual_embedding.py` is broken against the current code; it references a function that no longer exists.
- This documentation task changed **no** application source file.